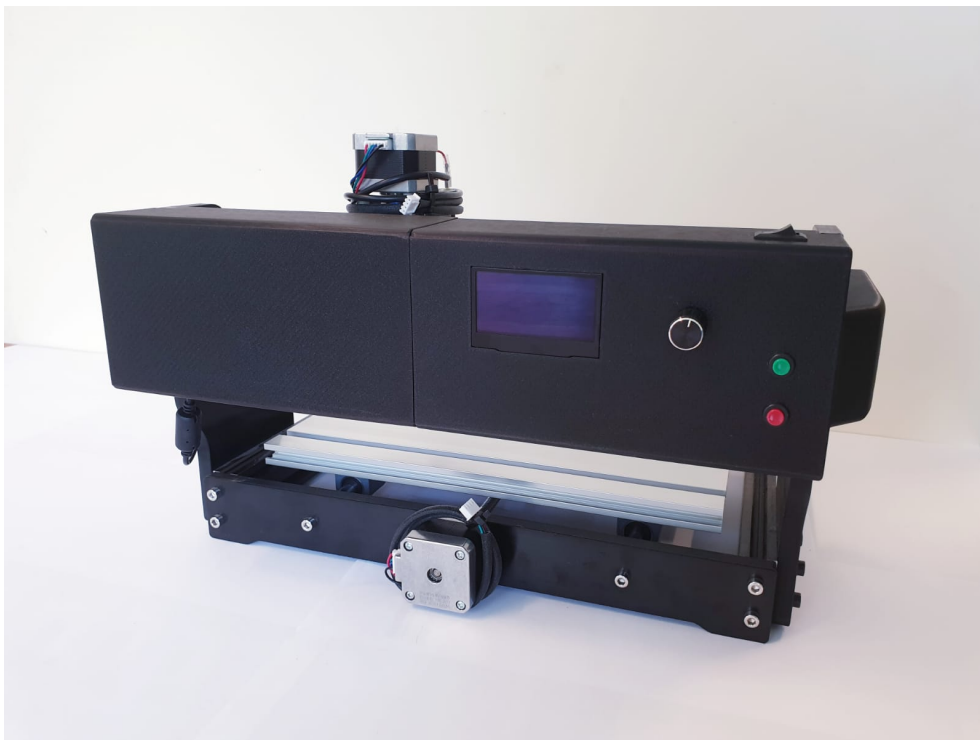




Chantal Crety, Jessica Perewersenko, Hannah Mey

Demonstrator Schrittmotor

Antrieb eines Schrittmotors mit dem Arduino Nano 33 BLE Sense

July 11, 2025

Contents

Contents	3
List of Figures	7
List of Listings	9
1. Projektbeschreibung	13
1.1. Einleitung	13
1.2. Zielsetzung	13
1.3. Umsetzung	13
1.4. Ergebnis	14
I. Arduino IDE	15
2. Beschreibung der Software IDE	17
2.1. Installation der Arduino IDE	17
2.2. Beschreibung der Entwicklungsumgebung	17
2.3. Erste Schritte in der Entwicklungsumgebung	17
2.3.1. Auswahl des Mikrocontrollers	17
2.3.2. Bibliotheken einbinden	18
2.4. Programmierung	18
2.4.1. header	18
2.4.2. setup()	18
2.4.3. loop()	18
2.5. Erster Programmtest	19
3. Arduino IDE 2.3.6	21
3.1. Beschreibung der Arduino IDE	21
3.2. Installation of the Arduino IDE	22
3.2.1. Download of the Arduino IDE	22
3.2.2. Summary of Options	22
3.2.3. Installation on Windows	23
3.2.4. Installation (MacOS)	25
3.3. Overview	27
3.4. Features	28
3.4.1. Sketchbook	28
3.4.2. Boards Manager	29
3.4.3. Library Manager	29
3.5. Integrating and Using a Custom Library in Arduino IDE	30
3.5.1. Creating the Library Files	30
3.5.2. Installation and Integration of the Library	30
3.5.3. Using Symbolic Links for Library Integration	30
3.5.4. Windows Tool <code>mklink</code> to Create Symbolic Links	30
3.5.5. MacOS Command <code>ln -s</code> to Create Symbolic Links	32
3.5.6. Verifying Library Availability in the Arduino IDE	32
3.5.7. Serial Monitor	33

3.5.8. Serial Plotter	33
3.6. Examples	33
3.7. Debugging	34
3.8. Autocompletion	34
3.9. Conclusion	35
4. To-Do	37
5. Setup for the Arduino Nano 33 BLE Sense	39
5.1. Introduction	39
5.2. Configuration for the Arduino Nano 33 BLE Sense	39
5.2.1. Installation of the driver	39
5.2.2. Connection and Configuration of the Arduino Board to the Computer	40
5.2.3. Select the Appropriate Port	40
5.2.4. Upload and Verify a Sketch	42
5.3. Test the Configuration	43
5.3.1. Testing the Steps by an Example Sketch blink.ino	43
5.3.2. Description of Basic Sketch for Printing 'Hello'	44
II. Arduino Nano 33 BLE Sense - Onboard Sensoren	47
6. Hardwarebeschreibung des Arduino	49
6.1. Aufbau des Arduinos	49
6.2. Integrierte Sensorik	50
6.2.1. 9-Achs-IMU für die Bewegungserkennung (LSM9DS1)	50
6.2.2. Näherungs-,Umgebungslicht-, Farb- und Gestensensor (APDS9960)	50
6.2.3. Barometrischer Drucksensor (LPS22HB)	51
6.2.4. Sensor für relative Luftfeuchtigkeit und Temperatur (HTS221)	51
6.2.5. Digitales Mikrophon (MP34DT05)	51
6.2.6. DC-DC-Wandler (MPM3610)	51
6.3. Beschreibung der Schnittstellen	52
6.3.1. I ² C	52
6.3.2. USB	53
6.3.3. Bluetooth®5	53
6.3.4. Weitere Kommunikationsschnittstellen	53
6.3.5. Digitale Ein- und Ausgangspins	53
6.3.6. Analoge Eingangspins	54
6.3.7. Weitere Pins	54
6.3.8. Reset-Knopf	54
6.3.9. LED-Lampen	54
6.4. Hinweis Arduino 33 BLE Sense Lite	55
6.5. Bezugsquellen	55
III. Arduino Nano 33 BLE Sense - External Sensors and Actors	57
7. Drehwinkel-Encoder	59
7.1. Encoder	59
7.2. Drehwinkel-Encoder	59
8. Schrittmotor NEMA 17	61
8.1. Beschreibung eines Schrittmotors	61
8.1.1. Aufbau und Funktionsweise eines Schrittmotors	61

Contents	5
8.1.2. Schrittmotor Bauformen	62
8.1.3. Betriebsarten unipolar und bipolar	63
8.1.4. Mikroschrittverfahren	63
8.2. Schrittverluste verhindern	64
8.2.1. Auswahl des Schrittmotors	64
8.2.2. Betriebsart	64
8.2.3. Externe Ereignisse	66
8.3. Beschreibung des verwendeten Schrittmotors	67
8.4. Spezifikation	67
9. Schrittmotortreiber DRV8825	69
9.1. Allgemeine Beschreibung eines Motortreibers	69
9.2. Spezifische Beschreibung des Schrittmotortreibers DRV8825	69
9.2.1. Anschluss des Treibers mit dem Arduino Nano 33 BLE Sense	69
9.3. Spezifikationen	70
9.4. Kalibrierung	70
9.5. Einfaches Beispiel mit Code	70
9.6. Weiterführende Literatur	71
10. Geschwindigkeitssensor LM393	73
10.1. Allgemeine Beschreibung des Sensors	73
10.2. Spezifische Beschreibung des Sensors	73
10.2.1. Funktionsweise der Lichtschranke	73
10.2.2. Signalverarbeitung mit dem LM393 Komparator	74
10.2.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense	74
10.3. Spezifikationen	74
10.4. Bibliothek	74
10.4.1. Installation	74
10.4.2. Code-Beispiel	75
10.5. Kalibrierung	76
10.6. Einfaches Beispiel mit Code	76
10.7. Tests	77
10.8. Weiterführende Literatur	78
11. OLED-Display	79
11.1. Allgemeine Beschreibung eines OLED-Displays	79
11.2. Spezifische Beschreibung OLED-Displays	79
11.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense	79
11.4. Spezifikationen	80
11.5. Bibliothek	80
11.5.1. Installation	80
11.5.2. Code-Beispiel	80
11.6. Einfaches Beispiel mit Code	80
11.7. Weiterführende Literatur	81
IV. Anhang	83
12. Materialliste	85
Literaturverzeichnis	89
Stichwortverzeichnis	92
Index	93

List of Figures

2.1. Hauptoberfläche in der Arduino IDE (Eigenaufnahme)	17
2.2. Installation des Boards (Eigenaufnahme)	18
2.3. Herunterladen von Bibliotheken (Eigenaufnahme)	18
3.1. Menu Button	21
3.2. Arduino IDE 2.3.3 Download	22
3.3. Summary of DownloadOptions	23
3.4. Arduino IDE Create Agent Installation	23
3.5. Arduino Setup Installation options	24
3.6. Arduino Setup Installation Folder	24
3.7. Arduino IDE Sketch	24
3.8. Steps for Installation	25
3.9. ArduinoIDE Create Agent Installation	25
3.10. Open the Download folder	26
3.11. Copy to the Applications Folder	26
3.12. Arduino IDE Sketch	27
3.13. Overview	28
3.14. Sketchbook	28
3.15. Board Manager	29
3.16. Library Manager	29
3.17. Command Prompt with Administrator Privileges	31
3.18. Command Prompt	31
3.19. Including the Nano33BLESenseLED Library	32
3.20. Serial Monitor	33
3.21. Examples	34
3.22. Debugging Example	34
3.23. Autocomplete	35
3.24. Menu Bar Option	35
5.1. Arduino Mbed OS Nano Boards Installation	40
5.2. Select the Connected board - here Arduino Nano 33 BLE Sense	40
5.3. Arduino Nano 33 BLE Sense Reset Button	41
5.4. green and orange Builtin-LED	42
5.5. Select Available Port for Uploading Arduino Sketch	42
5.6. Upload the Program in Arduino board	43
5.7. Setting the Port	43
5.8. LED Example Sketch	44
5.9. LED-Built Example	44
6.1. Arduino Nano 33 BLE Sense (Vereinfachte Darstellung)	49
6.2. Arduino Nano Pinbelegung [Ard24a]	52
8.1. Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]	62
8.2. Start-Stopp Frequenz [Fau20]	64
8.3. Trapezförmiges Geschwindigkeitsprofil [Fau20]	65
10.1. Aufrufen der Bibliotheken in der Arduino IDE	75

10.2. Installation der TimerOne Bibliothek	75
10.3. Aufrufen der Beispiele für die TimerOne Bibliothek	76
11.1. Installation der U8g2 Bibliothek	81

List of Listings

2.1. Blink.py	19
7.1. Testprogramm für den Encoder	60

Acronyms

AAR	Automatic Address Recognition
ADC	Analog to Digital Converter
AES	Advanced Encryption Standard
BLE	Bluetooth Low Energy
CCM	Counter with CBC-MAC
DMA	Direkt Memory Access
ECB	Electronic Codebook
EMI	Electromagnetic Interference
FPU	Floating Point Unit
I²C	Inter-Integrated Circuit
IC	Inter Circuit
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
LED	Light Emitting Diode
MUSB	Mini Universal Serial Bus
NFC	Near Field Communication
OLED	Organic Light Emitting Diode
PDM	Pulse Density Modulation
SCL	Serial Clock
SDA	Serial Data
SMD	Surface-Mounted Device
SoC	System on Chip
SPI	Serial Peripheral Interface
UV	Ultraviolettstrahlung

1. Projektbeschreibung

1.1. Einleitung

Im Rahmen dieses Projekts wurde ein vorhandener CNC-Tisch erweitert, um einen seiner Schrittmotoren als Demonstrator mit variabler Geschwindigkeitsregelung zu betreiben. Ziel war es, die technische Integration und Ansteuerung über einen Arduino Nano 33 BLE Sense umzusetzen und gleichzeitig eine benutzerfreundliche Bedienoberfläche zu schaffen.

1.2. Zielsetzung

Die Hauptaufgabe bestand darin, einen Schrittmotor des CNC-Tisches in mehreren Geschwindigkeitsstufen betreiben zu können. Die Steuerung sollte über einen Arduino Nano 33 BLE Sense erfolgen, wobei die Auswahl der Geschwindigkeit über Bedienelemente möglich sein sollte. Zusätzlich sollte ein optischer Geschwindigkeitssensor vom Typ LM393 die Drehzahl erfassen und zur Kontrolle auf einem Display ausgeben.

1.3. Umsetzung

Die Umsetzung des Projektes erfolgte in mehreren Phasen:

Elektronische Integration:

- Anschluss des A8825-Motortreibers an den Arduino
- Anbindung des LM393-Sensors zur Geschwindigkeitsmessung
- Testbetrieb auf einem separaten Testaufbau vor Montage am CNC-Tisch

Programmierung:

- Entwicklung eines Arduino-Codes zur Ansteuerung des Schrittmotors in verschiedenen Geschwindigkeitsstufen
- Implementierung der Auswertung des Sensorsignals zur Bestimmung der Drehzahl
- Bereitstellung einfacher Benutzerinteraktion (über Dreh-Encoder und OLED-Display)

Mechanische Montage:

- Befestigung der Bauteile direkt am CNC-Tisch
- Konstruktion und Montage einer Blende zur Aufnahme der Bedienelemente

Test und Inbetriebnahme:

- Durchführung von Funktionstests in allen Geschwindigkeitsstufen
- Abgleich der Sensordaten mit den programmierten Sollwerten
- Dokumentation der Ergebnisse

1.4. Ergebnis

Der Demonstrator zeigt erfolgreich den Betrieb eines CNC-Schrittmotors in mehreren konfigurierbaren Geschwindigkeitsstufen. Die Integration des Sensors erlaubt eine Live-Rückmeldung über die aktuelle Drehzahl. Die Bedienelemente sind über eine selbst konstruierte Blende zugänglich und ermöglichen eine einfache Steuerung durch den Nutzer.

Part I.

Arduino IDE

2. Beschreibung der Software IDE

2.1. Installation der Arduino IDE

Die Programmierung von Mikrocontrollern, wie dem verwendeten Arduino Nano 33 BLE Sense Lite, erfolgt unter Verwendung der Software Arduino IDE in der Version 2.3.2. Als Alternative kann auch die Entwicklungsumgebung Qt verwendet werden, welche die Programmierung in der Programmiersprache C++ ermöglicht. Vorteil der Arduino IDE besteht in ihren Funktionen, welche das Einbinden der Software auf dem Mikrocontroller vereinfachen. Die Software Arduino IDE kann über die offizielle Webseite von Arduino bezogen werden, welche unter der folgenden URL erreichbar ist: www.arduino.cc/software erreichbar ist. Auf dieser Plattform stehen diverse Versionen, für unterschiedliche Betriebssysteme, wie Windows, Linux und Mac OS X, sowie in verschiedenen Dateiformaten, zur Auswahl bereit [Ard24b]. Die Installation der Anwendung erfolgt durch die Ausführung der heruntergeladenen Datei sowie die Befolgung der Installationsanweisungen. Im Anschluss ist eine korrekte Konfiguration des verwendeten Entwicklungsboards sowie die Installation entsprechender Bibliotheken erforderlich.

2.2. Beschreibung der Entwicklungsumgebung

Nach dem Start der installierten Anwendung öffnet sich die Hauptoberfläche, erkennbar in Abbildung 2.1

Figure 2.1.: Hauptoberfläche in der Arduino IDE (Eigenaufnahme)

Die Hauptoberfläche ist mit einer eine Menüleiste ausgestattet, die verschiedene Menüs wie *Datei*, *Bearbeiten* und *Sketch* enthält. Diese Menüs ermöglichen das Bearbeiten und Öffnen von Sketches, also Programmen in der IDE, das Kompilieren und Hochladen von Code sowie das Verwalten von Bibliotheken. Zudem ist in der Menüleiste ein Hilfe-Menü integriert, das bei Fragen und Problemen Unterstützung bietet. Die Symbolleiste auf der linken Seite der Hauptoberfläche bietet Schaltflächen für häufig genutzte Funktionen wie das Verwalten von Sketches, Boards und Bibliotheken. Im Zentrum der Hauptoberfläche befindet sich ein Code-Editor, welcher die Bearbeitung der Sketches ermöglicht. Nach dem ersten Kompilieren eines Sketches erscheint direkt darunter ein Ausgabefenster, welches den Kompilierungs- und Hochladeprozess sowie eventuelle Fehler während der Kompilierung anzeigt. Dadurch können Probleme im Code schnell identifiziert werden.

2.3. Erste Schritte in der Entwicklungsumgebung

2.3.1. Auswahl des Mikrocontrollers

In der Entwicklungsumgebung können Sketche geöffnet und geschrieben werden. Um den Sketch kompilieren zu können, ist es erforderlich das korrekte Board auszuwählen.

Dafür muss zunächst das entsprechende Board installiert werden. Dies erfolgt über den *Boards-Manager*, erkennbar in Abbildung 2.2.

Figure 2.2.: Installation des Boards (Eigenaufnahme)

Im *Boards Manager* steht die Datei Arduino Mbed OS Nano in der aktuellen Version 4.1.1 zum herunterladen und installieren bereit. Im Anschluss kann unter *Select Board* das Board Arduino BLE Sense 33 ausgewählt werden [Ard24d]. Alternativ kann das Board über ein USB-Kabel angeschlossen werden. Unter *Select Board* wird bereits das korrekte Board und der entsprechende COM, also in diesem Fall der USB-Port, angeboten. Nach der Auswahl sind die Installationsanweisungen zu befolgen, um das Board zu installieren.

2.3.2. Bibliotheken einbinden

Von Arduino gibt es bereits viele offiziell unterstützte Bibliotheken. Um diese in einem Sketch nutzen zu können, ist zunächst eine Installation erforderlich. Dazu kann über den *Library Manager* die benötigte Bibliothek heruntergeladen und installiert werden, erkennbar in Abbildung 2.3. Die Suchfunktion hilft dabei, die korrekte Bibliothek zu finden [Ard24c].

Figure 2.3.: Herunterladen von Bibliotheken (Eigenaufnahme)

Nachdem die Bibliothek installiert wurde, kann sie im header in den Sketch eingebunden werden. Bei Verwendung von Bibliotheken, die nicht im *Library Manager* zu finden sind, besteht die Möglichkeit diese über *Sketch->Include Libraries->Add .ZIP Library...* einzubinden.

2.4. Programmierung

Bei einem neuen Sketch sind bereits die Funktionen *void setup()* und *void loop()* hinterlegt.

2.4.1. header

Im *header* werden die erforderlichen Bibliotheken hinterlegt und initialisiert. Gleichzeitig können hier auch Adressen von Peripheriegeräten hinterlegt und erste wichtige Variablen definiert werden.

2.4.2. setup()

Die *void setup()*-Funktion wird bei jedem Neustart oder Reset einmal ausgeführt [Ard24f]. In diesem Beispiel wird die serielle Kommunikation gestartet, aber auch verwendete Pins initialisiert und zugewiesen.

2.4.3. loop()

In der *void loop()*-Funktion findet der größte Teil des Programms statt. Der Inhalt dieser Funktion wird dauerhaft wiederholt, bis entweder kein Strom mehr an dem

Arduino anliegt, oder der Reset-Knopf gedrückt wird, wodurch zunächst erst die *void setup()*-Funktion wieder gestartet wird [Ard24e].

2.5. Erster Programmtest

Um die Funktionalität eines Systems zu testen, wird in der Regel zunächst ein simples Programm oder eine grundlegende Funktion getestet. In diesem Fall kann hier über *Datei -> Beispiele -> 01.Basics -> Blink* ein Sketch geöffnet werden, in dem die auf dem Arduino Nano 33 BLE Sense Lite aufgebrachte LED in einem fest gelegten Takt blinkt. Auf diese Weise kann die korrekte Auswahl des Boards und die Übertragung des Sketches auf den Mikrocontroller getestet werden. 2.1

```
void setup() {  
    pinMode(LED_BUILTIN, OUTPUT);  
}  
void loop() {  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
    digitalWrite(LED_BUILTIN, LOW);  
    delay(1000);  
}
```

Listing 2.1.: Blink.py

In der *void setup()*-Funktion wird in diesem Beispiel über den Ausdruck *pinMode(LED_BUILTIN, OUTPUT)* die auf dem Mikrocontroller verbaute LED in dem Sketch aufgerufen und eingebunden. In *void loop()* wird durch den Befehl *digitalWrite()* die LED eingeschaltet, indem eine Spannung an sie angelegt wird. Nach einer kurzen Verzögerung durch den gleichen Befehl wird die LED dann wieder ausgeschaltet, indem die anliegende Spannung gesenkt wird. Die Verzögerung kann mit dem *delay()*-Befehl eingestellt werden, indem der Wert in der Klammer angepasst wird. Dabei wird der Wert in Millisekunden angegeben.[Ard14]

3. Arduino IDE 2.3.6

In diesem Kapitel werden die Funktionen und die Benutzeroberfläche der Arduino IDE erklärt. Die einzelnen Komponenten werden erklärt und es wird aufgezeigt, wie man diese benutzt. Des Weiteren gibt es eine Installationsanleitung und es wird aufgezeigt, wie der Arduino Nano 33 BLE Sense mit der Arduino IDE verknüpft wird.

3.1. Beschreibung der Arduino IDE

Die Arduino IDE ist eine open-source Software zur Bearbeitung, Kompilierung und Übertragung von Codes auf Arduino-Module. Sie ist plattformübergreifend nutzbar, basiert auf Java und ist mit Windows, macOS und Linux kompatibel. Die Arduino IDE unterstützt verschiedene Arduino-Module, sowie die Programmierung in C und C++.

Die Programme, welche in der Arduino IDE geschrieben werden nennt man "Sketches" und können auf den Mikrocontroller übertragen werden.

The 6 buttons are present on top of the screen are as follows:



Figure 3.1.: Menu Button

-  The icon check mark is used to **Verify** the written code. After the code is entered, selecting this icon checks for any errors and confirms that the code is properly structured. This step ensures the code is ready for use with the hardware.
-  The icon **Upload** in the Arduino IDE compiles your code and uploads it to the connected board, making it ready to run.
-  The icon **Start Debugging** icon in the Arduino IDE initiates a debugging session, enabling you to analyze your code by setting breakpoints, stepping through execution, and inspecting variables on compatible boards.
-  The icon **Serial Plotter** in the Arduino IDE opens a tool that visualizes real-time data sent from the board over the serial connection, displaying it as graphs for easier analysis.
-  The icon **Serial Monitor** in the Arduino IDE opens a terminal to view and send text-based data over the serial connection, enabling communication with the connected board in real time.

WS:hier noch einmal d
letzten Grafiken in der
Auflistung nach links
verschieben

3.2. Installation of the Arduino IDE

3.2.1. Download of the Arduino IDE

The Arduino Nano 33 BLE Sense utilizes the Arduino Software **Integrated Development Environment (IDE)** for programming, which is the most widely used IDE for all Arduino boards and can be run both online and offline. This open-source Arduino IDE simplifies the process of writing code and uploading it to the board. Various versions of the software are available for each operating system (OS), including macOS, Linux, and Windows. The Arduino community also offers an online platform for coding and saving sketches in the cloud. This online Arduino editor is the latest version of the IDE, featuring all libraries and support for new Arduino boards. To access these software packages, visit the following Website: [Arduino-Software](#), to stay up to date, as there are daily updates available on the mentioned link.

The editor can be downloaded directly from the Arduino Software page with ease. The download options can be seen in Figure 3.2.

Downloads



 **Arduino IDE 2.3.3**

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux ApplImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Figure 3.2.: Arduino IDE 2.3.3 Download

3.2.2. Summary of Options

Below is a summary 3.3 of the available download options for the Arduino IDE, tailored to different operating systems and individual needs. Choose the appropriate version for the device to ensure compatibility and access to the latest features.

Summary of Options:

Operating System	Option	Description
Windows	Windows 10 and newer (64-Bit)	Direct download for Windows 10 and newer 64-bit versions, without using the MSI installer.
	MSI Installer	Easy installation through an <code>.msi</code> package.
	ZIP File	Portable, no installation required, just extract and run directly.
Linux	ApplImage 64-Bit (x86-64)	Portable, no installation required.
	ZIP File 64-Bit (x86-64)	Portable, extract and run directly.
macOS	macOS Intel (10.15: Catalina or newer)	For Intel Macs with macOS 10.15 or newer.
	macOS Apple Silicon (11: Big Sur or newer)	For Apple Silicon Macs (M1, M2) with macOS Big Sur 11 or newer.

Figure 3.3.: Summary of DownloadOptions

3.2.3. Installation on Windows

The installation process for Arduino IDE 2 on a Windows computer will be described step by step in the upcoming section. Following the installation instructions, the subsequent chapter will provide an overview of the IDE's features, including the Board Manager, Library Manager, Sketchbook, and more.

To install the Arduino IDE 2 on a Windows computer, simply run the file downloaded from the software page Arduino Software [Arduino-Software](#). Follow the installation steps, as shown in the Figure ??, to ensure correct installation.

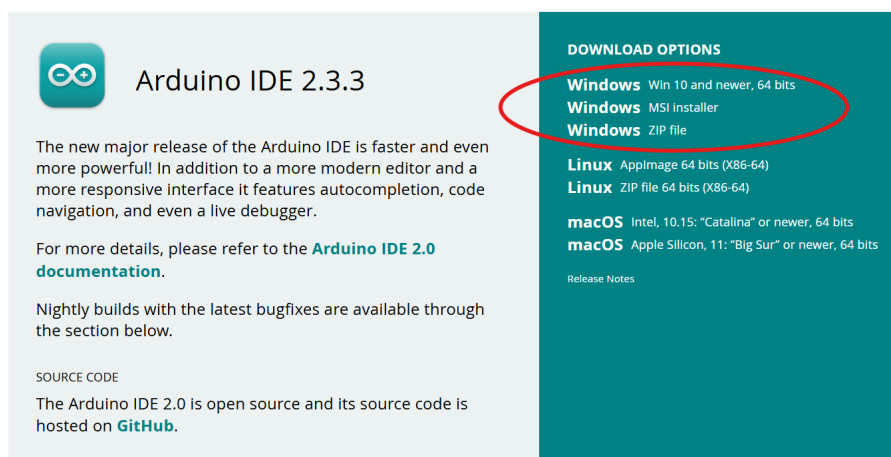
Downloads

Figure 3.4.: Arduino IDE Create Agent Installation

After the download is complete, open the `file setup` and proceed with the installation. Select all components as shown in Figure 3.5 in the dialog box, and then click `Next`.

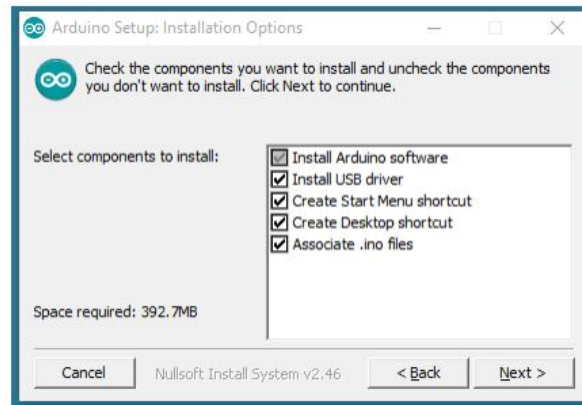


Figure 3.5.: Arduino Setup Installation options

Select the destination folder as seen in Figure 3.6 and click **Install**.

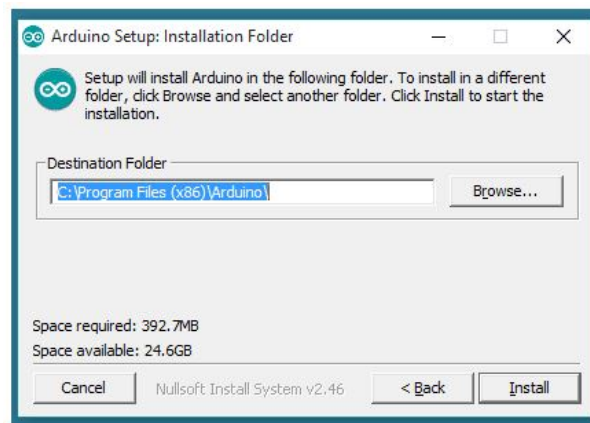


Figure 3.6.: Arduino Setup Installation Folder

Once the installation is complete, open the Arduino IDE. A default sketch will appear on the screen, as shown in Figure 3.7.

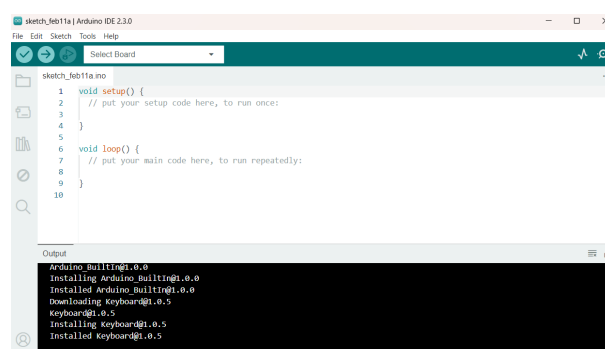


Figure 3.7.: Arduino IDE Sketch

It can be seen from the above figure 3.7 that the basic arduino sketch has two parts. The first part is the function `void setup()` which returns void and we do the initialization such as the output LED color, specifying the core etc. The second part is the function `void loop()` where we define functions which are to be performed

through out the loop. These codes are placed between paranthesis `{ }` and each function has a return type, here it has void return type.

Below in Figure 3.8 is a summary of the installation steps for the Windows version, outlining the key actions required to properly install the softwar

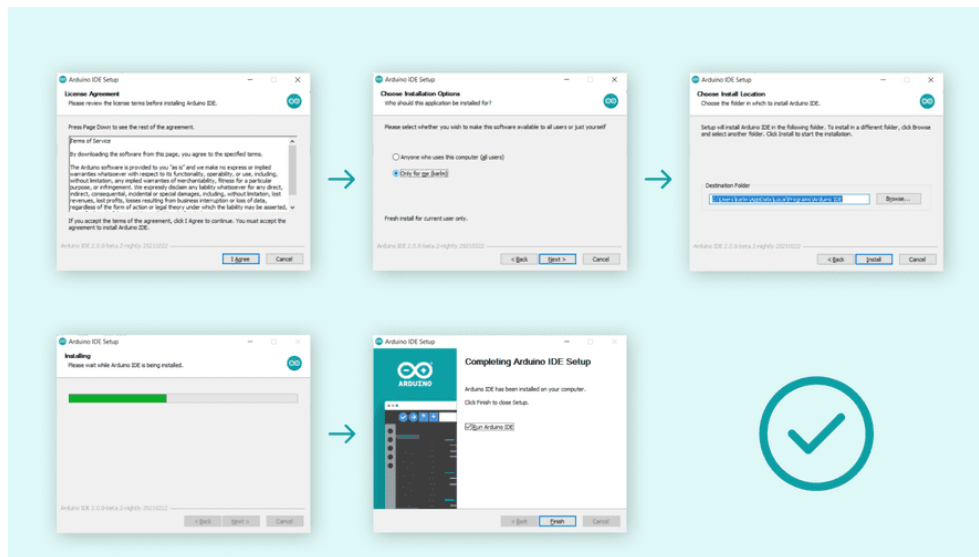


Figure 3.8.: Steps for Installation

3.2.4. Installation (MacOS)

To install the Arduino IDE, begin by downloading the latest version from the Arduino website Arduino Software [Arduino-Software](#). Select the version compatible with the specific operating system in use. In this case, Arduino 2.3.2 is being installed for macOS (Sonoma 14.4.1). The setup file is named [Arduino-Software](#) and has a size of 193,600 KB. This file is in Zip format. For those downloading Arduino IDE 2.3.X with Safari, the file will automatically extract upon download completion, while other browsers may require manual extraction. The figure below displays the latest offline version of Arduino IDE 2.3.3 3.9, which is also compatible with all operating systems.

Downloads



Arduino IDE 2.3.3

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux Appliance 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Figure 3.9.: ArduinoIDE Create Agent Installation

S:Die Installation muss aktualisiert werden, die folgenden Bilder müssen mit einem Mac ebenso aktualisiert werden

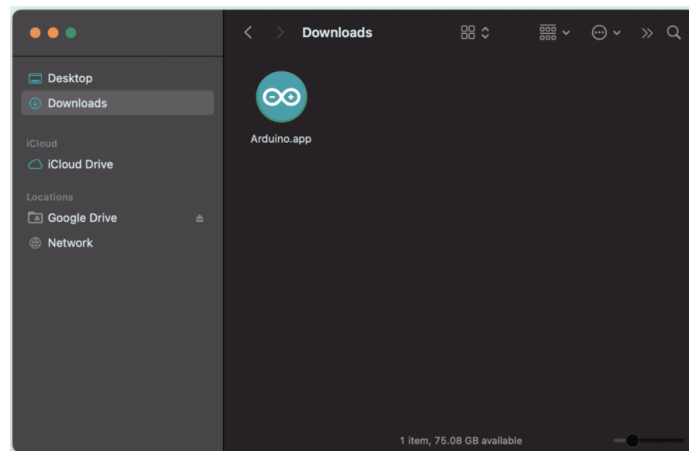


Figure 3.10.: Open the Download folder

Copy the Arduino application bundle into the application's folder (or elsewhere on the computer) then it looks like Figure 3.11.

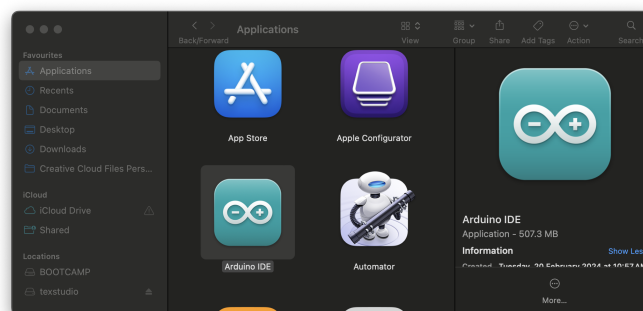


Figure 3.11.: Copy to the Applications Folder

It can be seen from the Figure 3.12 that the basic Arduino sketch has two parts.

- `void setup()`: This function returns void and performs initializations such as setting the output LED color and specifying the core.
- `void loop()`: In this function, specific operations to be executed within the loop are defined. The code for each operation is enclosed within curly braces `{ }`, and each function has a return type. In this case, the return type is `void`.

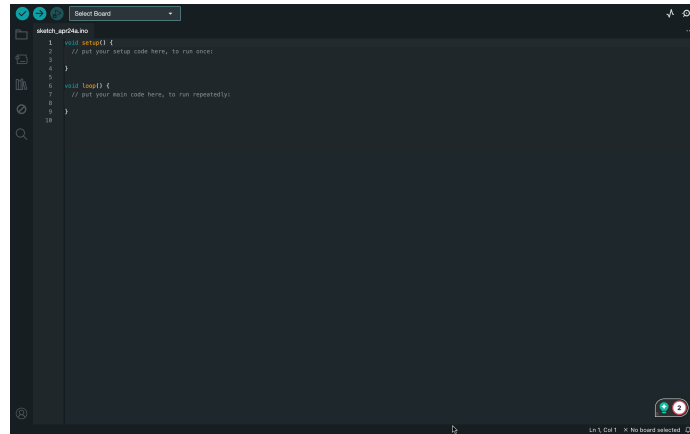


Figure 3.12.: Arduino IDE Sketch

WS:Figure 1.22 muss größer und alle Bilder in diesem Kapitel ohne Schwarzmodus

3.3. Overview

The Arduino IDE 2 features a new sidebar, making the most commonly used tools more accessible. 3.13 [Ard24]

- **Verify / Upload** These functions are used to compile and upload code to an Arduino board.
- **Select Board and Port** detected Arduino boards automatically show up here, along with the port number.
- **Sketchbook** This section serves as a centralized repository for all sketches that are stored locally on the user's computer. The Sketchbook is designed to provide a structured, easily accessible space for managing, organizing, and editing code developed within the Arduino environment. Additionally, the Sketchbook offers a synchronization feature with the Arduino Cloud, enabling seamless access to stored sketches across devices. This cloud integration allows users to retrieve and edit sketches from the online Arduino environment
- **Boards Manager** browse through Arduino and third party packages that can be installed. For example, using a MKR WiFi 1010 board requires the Arduino SAMD Boards package installed.
- **Library Manager** browse through thousands of Arduino libraries, made by Arduino and its community.
- **Debugger** test and debug programs in real time.
- **Search** search for keywords in the written code.
- **Open Serial Monitor** opens the Serial Monitor tool, as a new tab in the console.



Figure 3.13.: Overview

3.4. Features

The Arduino IDE 2 is a versatile development tool offering features like direct library installation, cloud synchronization for sketches, and built-in debugging tools. This section highlights some of its core features, with links to more detailed resources.

3.4.1. Sketchbook

The Sketchbook is where Arduino code files, known as sketches, are stored. These files are saved with the extension `.ino`. To maintain proper organization, each sketch must be placed in a folder named exactly the same as the sketch file. For example, a sketch named `MySketch.ino` must be saved in a folder named `MySketch`. This naming convention is essential for the Arduino IDE to correctly identify and manage the sketches. As seen in Figure 3.14.

Typically, sketches are stored in a folder named `Arduino` located within the Documents directory of the system.

To access the Sketchbook, the icon folder in the sidebar of the Arduino IDE can be selected. This action opens the directory where the sketches are stored, allowing for efficient management and access to the sketches.

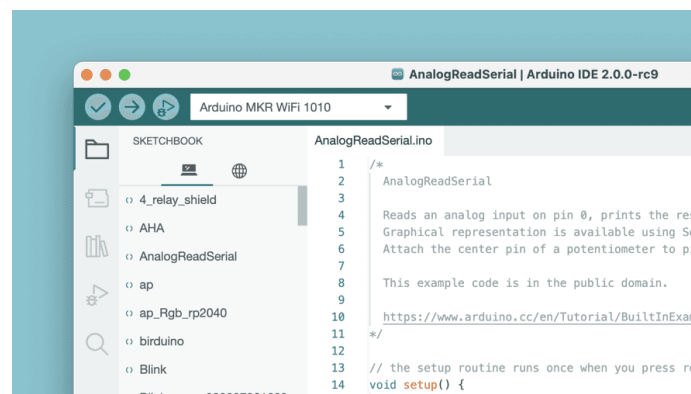


Figure 3.14.: Sketchbook

3.4.2. Boards Manager

The Board Manager can be found in the following steps: **Tools** » **Board** » **Board Manager**.

The Boards Manager in the Arduino IDE allows the installation of different board packages, as shown in Figure 3.15. A board package provides the necessary files and instructions to compile and upload code to specific types of Arduino boards.

Various board packages are available, such as avr, samd and megaavr. Each package supports different Arduino board families. For example, avr is for older boards like the Arduino Uno, while samd is used for newer boards like the Arduino Zero. Using the Boards Manager ensures that the right tools and files are installed for programming the selected board.

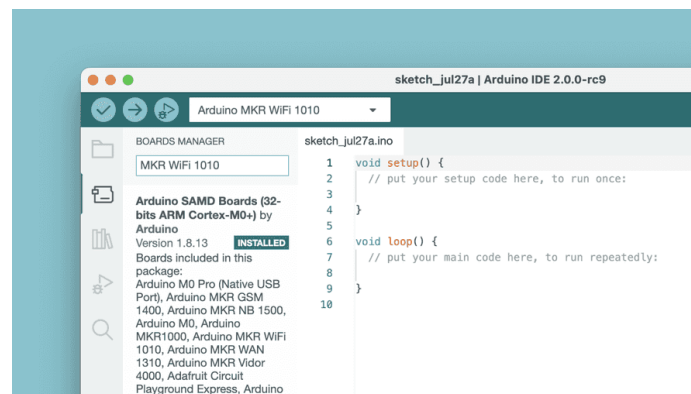


Figure 3.15.: Board Manager

3.4.3. Library Manager

The Library Manager can be found in the following steps: **Tools** » **Manage Libraries**.

The Library Manager in the Arduino IDE allows users to browse and install a wide range of libraries. Libraries extend the core Arduino functions, making it easier to perform tasks such as controlling servo motors, reading data from specific sensors, or working with modules like Wi-Fi. These libraries provide prewritten code to handle specific hardware components and simplify complex tasks, allowing for faster and more efficient development in Arduino projects. As seen in Figure 3.16.

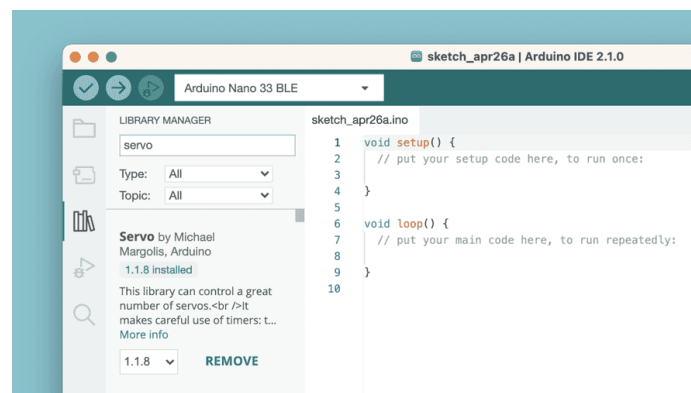


Figure 3.16.: Library Manager

3.5. Integrating and Using a Custom Library in Arduino IDE

In the development of embedded systems using the Arduino platform, libraries are essential for abstracting and simplifying complex functionality. A custom library is particularly beneficial when a specific functionality or hardware interface is repeatedly used across different projects. This section outlines the process of creating, installing, and using a custom library within an Arduino project, in the Arduino Integrated Development Environment (IDE).

3.5.1. Creating the Library Files

A well-structured custom Arduino library typically consists of at least two key components:

Header File (.h): This file contains the declarations of classes, functions, and variables that define the interface of the library. It provides the necessary definitions for the functions and classes to be used by external programs.

Source File (.cpp): This file contains the actual implementation of the functions and methods declared in the header file. It defines the behavior of the functions and classes. The library structure is organized in a way that allows for clear separation between the declaration and implementation of its functionalities.

3.5.2. Installation and Integration of the Library

Once the custom library has been created, it must be properly installed and integrated into the Arduino IDE to be used in projects.

To install a custom library in Arduino follow these steps:

1. Locate the Libraries Folder: The Arduino IDE searches for libraries in the libraries folder, which is located within the Arduino sketchbook directory. The typical path to this directory is:

```
C:/Users/<username>/Documents/Arduino/libraries
```

2. Copy the custom library folder, e.g., `Nano33BLESenseLE`, into the directory `libraries`. The final path should look like this:

```
C:/Users/<username>/Documents/Arduino/libraries/Nano33BLESenseLED
```

3. Restart the Arduino IDE: After placing the library in the correct directory, restart the Arduino IDE to ensure that it recognizes the newly added library.

3.5.3. Using Symbolic Links for Library Integration

In some cases, it may be more convenient or necessary to store the library files outside the default library directory. For example, if the libraries are stored in a GitHub repository or for better version control, the command `mklink` can be used to create a symbolic link. This allows the Arduino IDE to access the library files as if they were in the default directory, without duplicating the files.

3.5.4. Windows Tool `mklink` to Create Symbolic Links

Symbolic links allow libraries to be stored in a different location while still being treated by the Arduino IDE as if they reside in the standard directory `libraries`. This can be particularly useful for version-controlled environments, such as when using Git, or to organize libraries without duplicating files.

Steps for Creating a Symbolic Link:

1. **Open Command Prompt with Administrator Privileges:** Press **Win + S** and search for **cmd**, click on **Command Prompt** and select **Run as Administrator** as seen in Figure 3.17

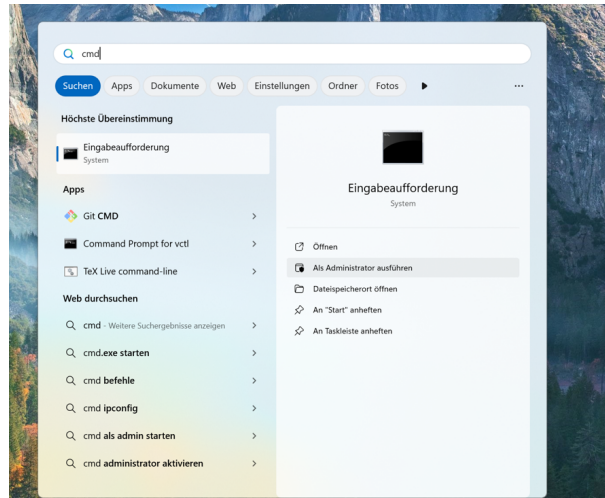


Figure 3.17.: Command Prompt with Administrator Privileges

2. **Execute the Command `mklink` :** The command for creating a symbolic link is: `mklink /D "TargetPath" "SourcePath"`

- **TargetPath:** The location where the Arduino IDE expects the library to be, here `libraries`.
- **SourcePath:** The actual location of the library.

For this example, the library is located at:

`C:/Users/MSRLabor/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/`

The target path in the Arduino IDE's libraries folder is:

`C:/Users/MSRLabor/Documents/Arduino/libraries/Nano33BLESenseLED`

The full command can be seen in Figure 3.18.

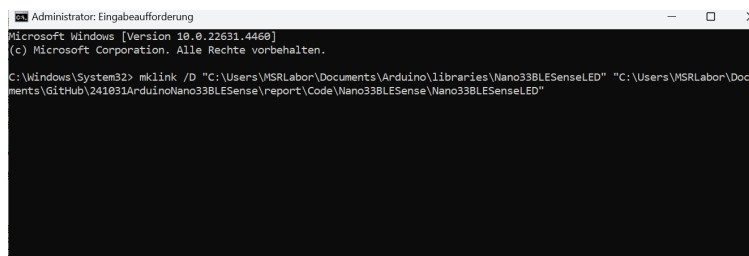


Figure 3.18.: Command Prompt

Verifying the Symbolic Link: After executing the command, navigate to the folder `libraries`. A folder named `Nano33BLESenseLED` should be present, acting as a symbolic link pointing to the actual library location.

WS:Figure 1.17, 1.18 w
an english sytem, so th
figures are on english

3.5.5. MacOS Command `ln -s` to Create Symbolic Links

1. Open Terminal: Open the `Terminal` application by searching for it in `Applications`, go to `Utilities`.
2. Execute the Command `ln -s` : The command for creating a symbolic link is:
`ln -s "SourcePath" "TargetPath"`
 - **TargetPath:** The location where the Arduino IDE expects the library to be, here `libraries`.
 - **SourcePath:** The actual location of the library.

For this example, the library is located at:

`C:/Users/MSRLabor/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/`

The target path in the Arduino IDE's libraries folder is:

`C:/Users/MSRLabor/Documents/Arduino/libraries/Nano33BLESenseLED`

The full command would be:

`ln -s "/Users/username/Documents/GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/"
"/Users/username/Documents/Arduino/libraries/Nano33BLESenseLED"`

Verifying the Symbolic Link: After executing the command, navigate to the folder `libraries`. A folder named `Nano33BLESenseLED` should be present, acting as a symbolic link pointing to the actual library location.

WS:here are some
pictures necessary as in
1.5.4

3.5.6. Verifying Library Availability in the Arduino IDE

To ensure that the custom library has been successfully integrated and is recognized by the Arduino IDE, follow these steps:

1. Restart the Arduino IDE: If the Arduino IDE was open during the library installation or after creating the symbolic link, close and reopen it. This step ensures the IDE reloads its list of available libraries.
2. Check the Library Menu: Navigate to `Sketch` go to `Include Library` and select the Library `Nano33BLESenseLED`, as seen in Figure 3.19
3. Verify Inclusion in the List: If the library is listed, it indicates successful recognition by the IDE. If not, double-check the directory structure and symbolic link to confirm they are correctly configured.
4. Compile a Test Program: Write a simple test program using functions or classes from the library. Compile the sketch to ensure there are no errors, which confirms that the library has been successfully integrated.

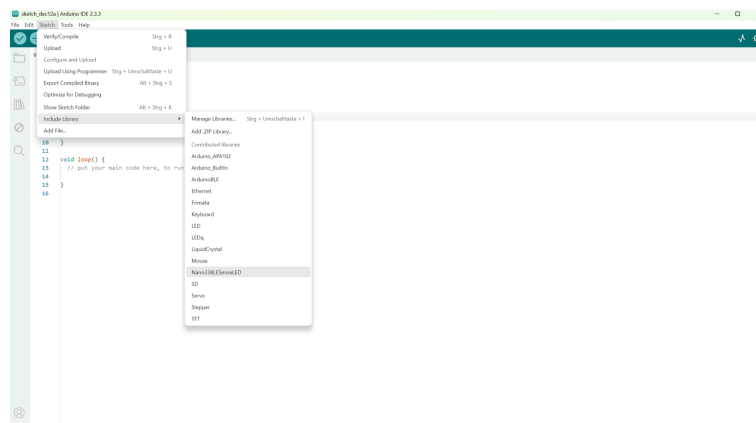


Figure 3.19.: Including the Nano33BLESenseLED Library

Once the program compiles without errors, the library is ready to use, and you can confidently include it in your Arduino projects.

3.5.7. Serial Monitor

The Serial Monitor can be found in the following steps: **Tool** **Serial Monitor**

The Serial Monitor is a tool that allows to view data streaming from the board, via for example the command `Serial.print()`.

Historically, this tool was located in a separate window but is now integrated with the editor, making it easy to run multiple instances simultaneously on your computer. The Serial Monitor can be seen in Figure 3.20.



Figure 3.20.: Serial Monitor

3.5.8. Serial Plotter

The tool Serial Plotter is great for visualizing data with graphs and monitoring, such as voltage peaks. It can monitor multiple variables simultaneously, with the option to enable specific types.

3.6. Examples

A significant component of the Arduino Documentation is the set of example sketches bundled with various libraries. These examples demonstrate the practical application of library functions, showcasing their intended uses and main features. Libraries included with certain board packages may also contain their own example sketches. To access these example sketches whether from libraries installed manually or those included with board packages navigate to **File** **Examples**, and locate the desired library from the list.

For instance, when an UNO R4 WiFi board is connected, the examples list appears with options specific to that board. An example sketch demonstrating a pre-loaded Tetris animation can be accessed by navigating to **File** **Examples** **LED Matrix** **MatrixIntro** and uploading it to the connected board. This example shows how the board's bundled code can be used in practice, assisting users in learning how to control various hardware functions directly.

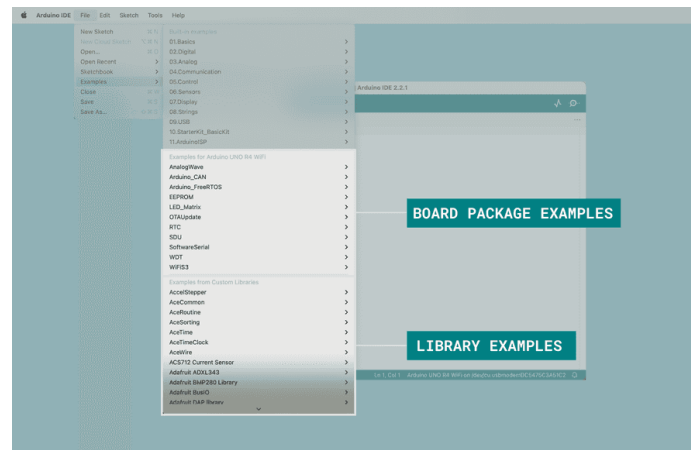


Figure 3.21.: Examples

In Figure 3.21 above, the examples list is shown when a UNO R4 WiFi board is connected to the computer.

3.7. Debugging

The tool Debugger in Arduino IDE 2.3.3 is designed to assist in testing and debugging programs by providing the ability to step through code execution in a controlled manner. This tool allows users to set breakpoints, step through code line-by-line, and inspect variables, helping to identify issues such as logic errors or incorrect variable values. The debugger enables users to pause execution at specific points in the program, allowing for a detailed examination of the program's behavior and memory state. This feature enhances the development process by making it easier to locate and fix errors during runtime, rather than relying solely on print statements or trial and error.

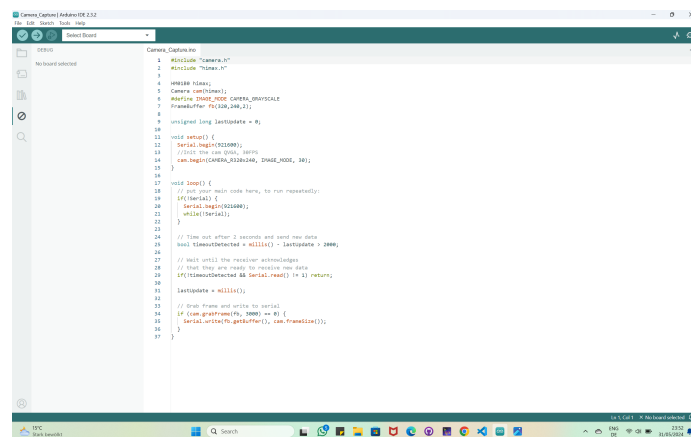


Figure 3.22.: Debugging Example

S:Debugging Bild muss
setzt werden, weil nicht
zu lesen

3.8. Autocompletion

Autocompletion is a key feature in Arduino IDE version 2, making it easier to code by suggesting functions and elements from the Arduino API. This feature helps identify and complete code more quickly, improving efficiency and accuracy.

For autocompletion to work, the board must be selected in the IDE. This ensures that the editor recognizes the correct functions and libraries for the specific board, allowing accurate suggestions. 3.23

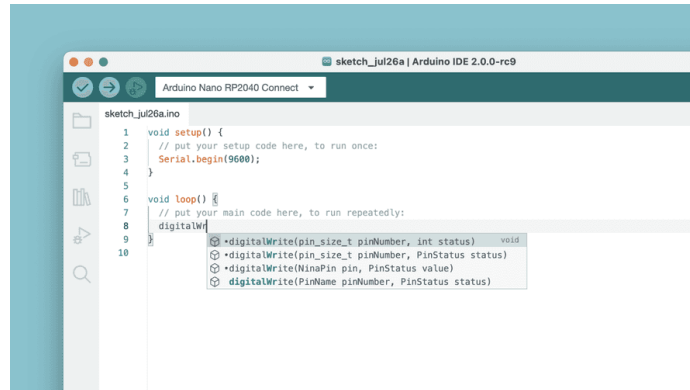


Figure 3.23.: Autocomplete

3.9. Conclusion

This guide provides an overview of key features in Arduino IDE 2, along with references to detailed articles for further exploration. Each section aims to enhance understanding and enjoyment of the wide range of functionalities included in the IDE.

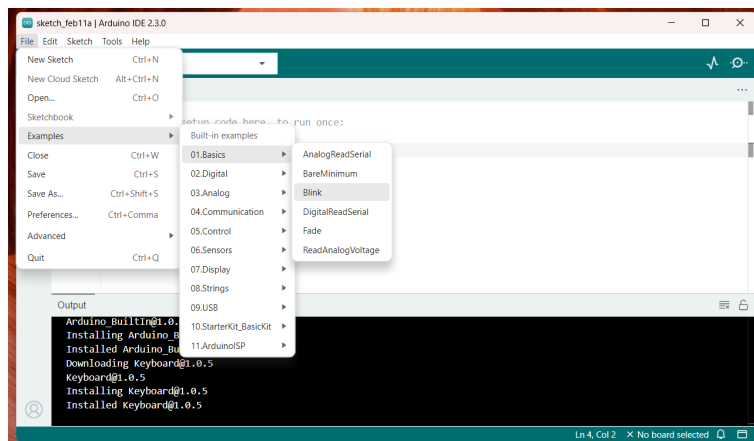


Figure 3.24.: Menu Bar Option

4. To-Do

- Mac Instalation nochmal durchführen und aktualisieren (gilt auch für die Bilder)
- In Discription: Item-Punkte anpassen mit den Bildsymbolen
- MyNote Kommentare beachten

5. Setup for the Arduino Nano 33 BLE Sense

5.1. Introduction

In this chapter, the process of connecting the Arduino Nano 33 BLE Sense to a computer or laptop and configuring essential settings to get started is explored. The steps for installing the necessary tools and ensuring the board is correctly set up in the Arduino IDE are detailed. Finally, a simple example sketch is demonstrated to verify that the board is working as expected and ready for further development.

There are set of examples which are build in Arduino (IDE) for the testing purpose, for checking all the configuration and setting up the board we can open one of the basic example [blnik.ino](#) first.

5.2. Configuration for the Arduino Nano 33 BLE Sense

To program the **Arduino Nano 33 BLE Sense** in an offline environment, follow these steps:

5.2.1. Installation of the driver

1. **Installation of the driver:** Begin by installing the latest version of the Arduino IDE on your computer. Refer to Section ?? for more details.
2. **Installation of the packages:** Once the IDE installation is complete, open the `Arduino IDE` and navigate to the menu `Tools`, located in the upper-left corner.
3. In the menu `Tools`, select the `Board Manager`.
4. In the window `Board Manager`, use the search bar to locate the **Arduino Nano 33 BLE Sense** by typing its name as shown in Figure 5.1 .
5. From the search results, select `Arduino Mbed OS Nano Boards` and click `Install` to install the required package.

The Mbed OS Nano Board package includes support for several Arduino Nano family boards, including the Arduino Nano 33 BLE Sense. After completing the installation, connect the Arduino Nano 33 BLE Sense to your computer using a USB-A to USB-Micro to begin programming.

1. Connect the Arduino board to the computer using a USB-A-USB-micro-cable.
2. Open the Arduino IDE on the computer. A blank environment page will appear, showing the default `void setup()` and `void loop()` functions.
3. Navigate to the menu `Tools`, select `Board`, and choose the connected board, which is the **Arduino Nano 33 BLE Sense**, as shown in Figure 5.2.

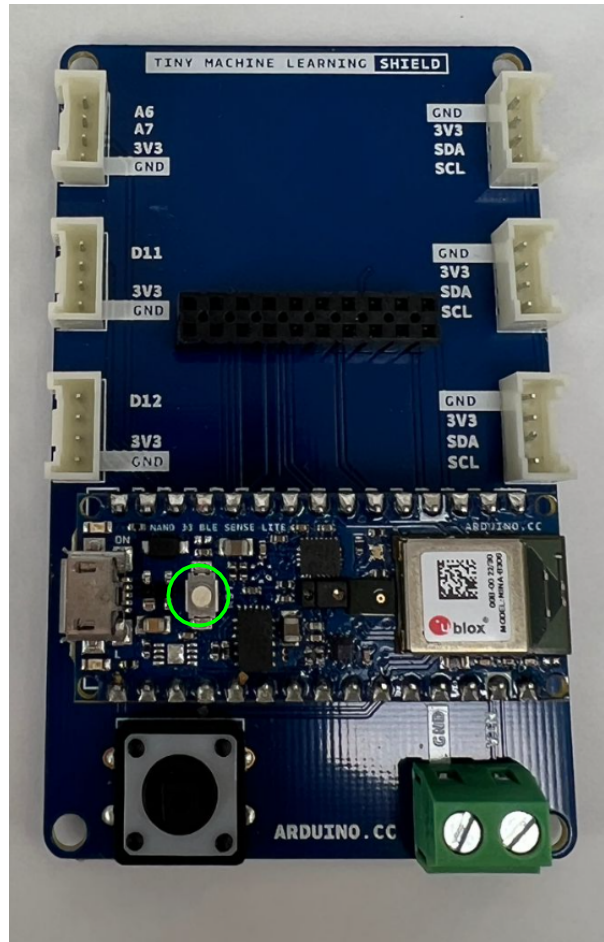


Figure 5.3.: Arduino Nano 33 BLE Sense Reset Button

By pressing the white reset button, the Arduino board will enter Bootloader mode. It is important to verify that the orange Built-in-LED is illuminated, as shown in Figure 5.4

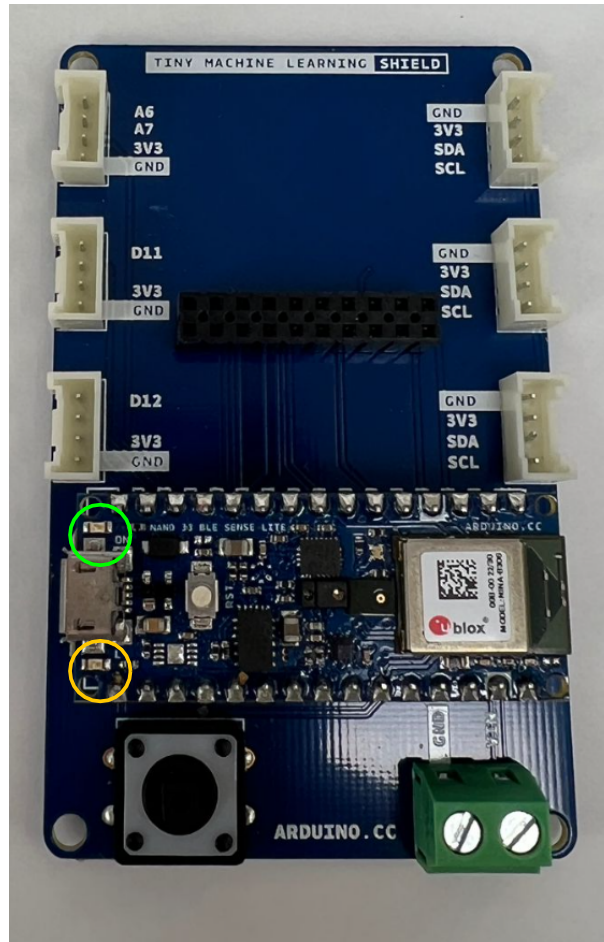


Figure 5.4.: green and orange Builtin-LED

After successfully completing the previously mentioned steps, the next task is to select the connected port before uploading the program. To do this, navigate to the menu **Tools**, select **Port**, and ensure that the available port for uploading the program is properly selected, as shown in Figure 5.5

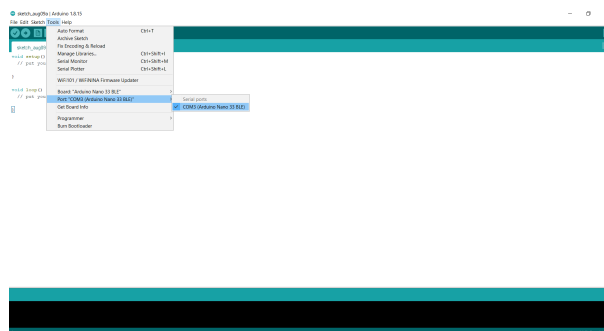


Figure 5.5.: Select Available Port for Uploading Arduino Sketch

5.2.4. Upload and Verify a Sketch

After ensuring that the appropriate port is selected, the next step is to upload the Arduino program. **Before** uploading the program, it is considered best practice to

verify it first. This process will indicate whether any errors or warnings exist in the program. Once the program is successfully verified, it can be safely uploaded by clicking the button **Upload** located below the file section, as shown in Figure 5.6. After selecting the board and port, confirm that the Arduino Nano 33 BLE Sense is properly connected to the Arduino IDE. This can be verified by checking the message in the bottom right corner of the IDE window, which should display the connected board and port.



Figure 5.6.: Upload the Program in Arduino board

After uploading, the code will be compiled, and any issues in the program will be displayed in the bottom black window. Once the code is successfully uploaded and compiled on the Arduino board, it is necessary to reselect the port, as done previously. Navigate to the **Tools menu**, select **Port**, and ensure the correct port is selected, as shown in Figure 5.7, in order to view the output in the Serial Monitor.

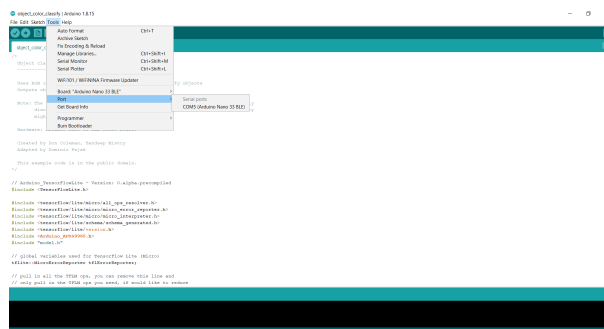


Figure 5.7.: Setting the Port

5.3. Test the Configuration

In this subsection, we will test whether the board is properly connected to the Arduino IDE by running an Example Blink Sketch. This simple test will make an onboard LED light up, verifying both the connection and basic functionality of the Arduino Nano 33 BLE Sense. By the end of this section, you will have confirmed that the hardware and software are working together seamlessly.

5.3.1. Testing the Steps by an Example Sketch `blink.ino`

The Arduino IDE contains a set of built-in examples for testing purposes. To verify the configuration and set up the board, the basic example `blink.ino` can be opened first. After uploading the example `blink.ino` the Builtin LED blinks with 2 Hz. To begin, open the Arduino IDE on the computer and follow the steps below:

1. **Open the Examples:** Once it's open, click on the **File** menu and hover over **Examples** in the dropdown menu.
2. **Selecting the Example Sketch:** A list of example categories will appear. Under the Built-in Examples, go to **01.Basics** and click on **Blink** as shown in 5.8.
3. **Blink Example Sketch:** After following steps 1 and 2, the Example Sketch **blink.ino** will open in a new window within the IDE as seen in Figure 5.9.
4. **Upload Example Sketch **blink.ino**:** Click the **Upload** button (right arrow icon) in the Arduino IDE toolbar to compile and upload the example sketch to the Arduino Nano 33 BLE Sense board. Once the upload is complete, the orange built-in LED starts blinking.

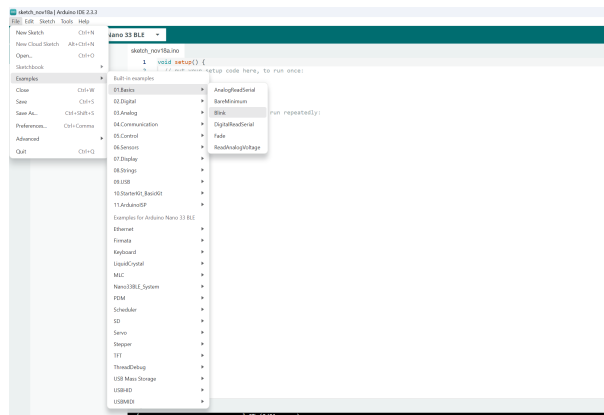


Figure 5.8.: LED Example Sketch

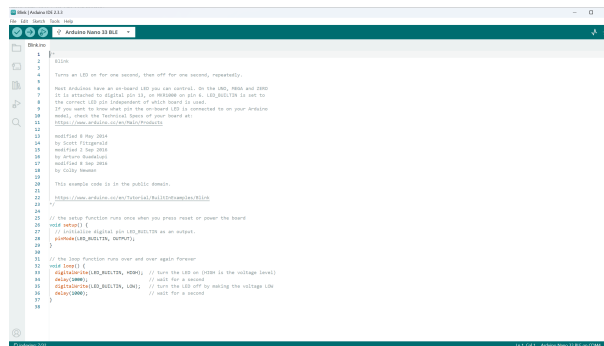


Figure 5.9.: LED-Built Example

With the successful blinking of the built-in orange LED, it is confirmed that the Arduino Nano 33 BLE Sense is properly connected to the Arduino IDE and its basic functionality has been verified. The board is now ready for further development and projects.

5.3.2. Description of Basic Sketch for Printing 'Hello'

To test the development environment and the basic functionality of the hardware, a simple example sketch that controls only one LED is suitable. This sketch provides the Arduino Integrated Development Environment (IDE). Under the path **File/Examples/01.Basics**, small example sketches can be selected. Here, the example **Fade** is used. In this case, the built-in LED, which is an RGB LED, is utilized.


```
int brightness = 0;  // how bright the LED is
int fadeAmount = 5;  // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
    // declare LED_BUILTIN to be an output:
    pinMode(LED_BUILTIN, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
    // set the brightness of LED_BUILTIN:
    analogWrite(LED_BUILTIN, brightness);

    // change the brightness for next time through
    //the loop:
    brightness = brightness + fadeAmount;

    // reverse the direction of the fading at the ends
    //of the fade:
    if (brightness <= 0 || brightness >= 255) {
        fadeAmount = -fadeAmount;
    }
    // wait for 30 milliseconds to see the dimming
    //effect
    delay(30);
}
```

As a result, the brightness of the built-in LED should gradually fade in and out.

Part II.

**Arduino Nano 33 BLE Sense -
Onboard Sensoren**

6. Hardwarebeschreibung des Arduino

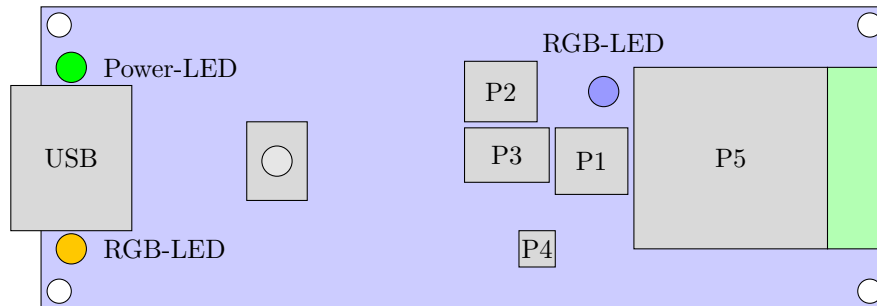


Figure 6.1.: Arduino Nano 33 BLE Sense (Vereinfachte Darstellung)

Pos.Nr.	Modul/Sensor
P1	Mikrophone
P2	9-Achs-IMU
P3	Näherungs-,Umgebungslicht-, Farb- und Gestensensor
P4	Drucksensor
P5	Bluetooth-Modul

Table 6.1.: Arduino Nano 33 BLE Sense [Ard24a]

6.1. Aufbau des Arduinos

Der Arduino Nano 33 Bluetooth Low Energy (BLE) Sense ist ein Mikrocontroller basierend auf dem Nordic nRF52840-System on Chip (SoC). Trotz seiner kompakten Bauweise mit einer Größe von 45 x 18 mm verfügt es über mehrere verschiedene integrierte Sensoren, Aktoren und Anschlussschnittstellen (siehe Figure 6.1). Darüber hinaus verfügt der Arduino-Prozessor über 1 MB Flash-Speicher und 256 KB RAM. Bei dem Prozessor handelt es sich um einen Arm®Cortex-M4F Prozessorkern, welcher mit einer Taktrate von 64 MHz arbeitet und für Mikrocontroller Anwendungen optimiert ist. Dieser bietet eine gute Leistung bei geringen Stromverbrauch und ist auch für Echtzeit-Verarbeitungsaufgaben geeignet. Außerdem besitzt der Prozessor eine Floating Point Unit (FPU), die eine effiziente Abarbeitung von Datenverarbeitungsoperationen ermöglicht. Mit Hilfe einer integrierten Steckverbindung (engl. „header“) kann der Mikrocontroller direkt auf ein Board gesteckt werden. Der Name Arduino Nano BLE 33 Sense gibt bereits Aufschluss über einiger Eigenschaften und Funktionen des Mikrocontrollers:

- "Nano": steht für die kleine Bauform
- "33": bedeutet, dass das Board mit 3,3 Volt betrieben wird
- "BLE": zeigt die Unterstützung für Bluetooth Low Energy
- "Sense": weist auf die integrierten Sensoren hin.

[Arm20] [Ard24a]

6.2. Integrierte Sensorik

Der Arduino Nano 33 BLE-Sense verfügt über eine Vielzahl von integrierten Sensoren und Aktoren, mit denen verschiedene Umgebungseigenschaften erfasst werden können. Dazu gehören die in den folgenden Abschnitten beschriebenen Sensoren und Aktoren.

6.2.1. 9-Achs-IMU für die Bewegungserkennung (LSM9DS1)

Bei der Inertialmesseinheit LSM9DS1 handelt es sich um ein System-in-Package, d.h. es sind mehrere elektronische Komponenten bzw. Chips auf kleinstem Raum in einem Gehäuse kombiniert.[LD12] Die Inertial Measurement Unit (IMU) verfügt über einen 3D-Linearbeschleunigungsmesser, ein 3D-Gyroskop und einen 3D-Magnetometer. Darüber hinaus verfügt das System über eine serielle Inter-Integrated Circuit (I²C)-Busschnittstelle, die einen Standard und einen Fast Mode (100/400 kHz) bereitstellt, zudem eine serielle SPI-Standardschnittstelle. Der Sensor verfügt über einen linearen Beschleunigungsmesser (Accelerometer) mit wählbarer Skala von $\pm 2/\pm 4/\pm 8/\pm 16$ g, der die lineare Beschleunigung in drei Achsen (x, y, z) misst und die Erfassung von Geschwindigkeits- und Positionsänderungen ermöglicht. Das Magnetfeld ist mit einer wählbaren Skala von $\pm 4/\pm 8/\pm 12/\pm 16$ Gs ausgestattet, damit misst es das magnetische Feld in den drei Achsen und ermöglicht die Bestimmung der Ausrichtung relativ zur Erdmagnetfeldrichtung. Das 3D-Gyroskop ist mit einem wählbarem Skalenendwert von: $\pm 125/\pm 250/\pm 500/\pm 1000/\pm 2000 \frac{\text{Grad}}{\text{s}}$ ausgestattet und ist für die Erfassung der Winkelgeschwindigkeit bzw. Drehbewegung um die drei Achsen zuständig.[STM15][Ard24a] Mögliche Anwendungsbereiche sind zum Beispiel eine Bewegungssteuerungen (Drohnensteuerung, Robotik und industrielle Automatisierung), Schwingungsüberwachung und -kompensation, Antennen, Plattformen, optische Bild- und Objektivstabilisierung.

6.2.2. Näherungs-,Umgebungslicht-, Farb- und Gestensensor (APDS9960)

Hierbei handelt sich um einen sehr vielseitigen Sensor. Er dient zur Gestenerkennung, Farberkennung, Abstandsmessung und Umgebungslichtmessung. Die Gestenerkennung nutzt vier gerichtete Fotodioden, um reflektierte Infrarot-Energie (von der integrierten Light Emitting Diode (LED)) zu erfassen, um physikalische Bewegungsinformationen (d.h. Geschwindigkeit, Richtung und Entfernung) in digitale Informationen zu übersetzen. Die Näherungserkennung ermöglicht die Messung der Entfernung durch die Erkennung der reflektierten Infrarot-Energie, (von der integrierten LED) mit Hilfe einer Fotodiode. Erkennungsereignisse sind interruptgesteuert und treten immer dann auf, wenn das Näherungsergebnis die oberen und/oder unteren Schwellenwerte überschreitet. Der Sensor verfügt außerdem über Offset-Einstellregister zur Kompensation des System-Offsets, der durch unerwünschte Infrarot-Energiereflexionen am Sensor verursacht wird. Die Intensität der Infrarot-LED wird werkseitig eingestellt, so dass eine Kalibrierung der Endgeräte aufgrund von Bauteilschwankungen nicht erforderlich ist. Die Farb- und Umgebungslichterkennung liefert Daten zur roten, grünen, blauen Farbspektrum und Daten zum klaren Lichtintensität. Jeder der Kanäle R, G, B, C verfügt über einen Ultraviolettstrahlung (UV)- und Infrarot-Sperrfilter und einen speziellen Datenkonverter, der gleichzeitig 16-Bit-Daten erzeugt. Diese Architektur ermöglicht Anwendungen eine genaue Messung des Umgebungslichts zu messen und die Farbe zu erkennen, was es den Geräten wiederum ermöglicht, die Farbtemperatur zu berechnen und die Hintergrundbeleuchtung des Displays dementsprechend anzupassen.[Ava15][Ard24a]

6.2.3. Barometrischer Drucksensor (LPS22HB)

Der LPS22HB ist ein sehr kompakter Absolutdrucksensor, der auf dem piezoresistiven Prinzip basiert. Er arbeitet als Barometer und verfügt über einen digitalen Ausgang. Zudem ist in diesem Drucksensor ein Temperatursensor integriert, mit welchem Druckmessungen zusätzlich kompensiert werden können. Das Gerät besteht aus einem Sensorelement und einer Inter Circuit (IC)-Schnittstelle, die eine Kommunikation zwischen der Sensoreinheit und der Anwendung über I²C oder Serial Peripheral Interface (SPI) ermöglicht. Das Sensorelement erfasst den absoluten Druck und besteht aus einer speziell gefertigten, hängenden Membran. Der LPS22HB ist in einem vollständig vergossenem Land-Grid-Array-Gehäuse untergebracht, das kleine Löcher aufweist. Durch diese Öffnung gelangt der externe Druck auf das Sensorelement. Der Betrieb des Sensors ist über einen Temperaturbereich von -40 °C bis +85 °C gewährleistet. [STM17][Ard24a] Anwendungsbereiche für diesen Sensor sind zum Beispiel: Wetterstationen, Höhenmesser, Luftdrucküberwachung in industriellen Prozessen oder tragbare smarte Geräte, wie Sportuhren oder Smartphones.

6.2.4. Sensor für relative Luftfeuchtigkeit und Temperatur (HTS221)

Der HTS221 ist ein kompakter Sensor zur Messung von relativer Luftfeuchtigkeit und Temperatur. Er enthält ein Sensorelement und eine Mischsignalverarbeitungseinheit, welche die gemessenen Daten über digitale Schnittstellen bereitstellt.

Der Sensor kann über I²C und auch über SPI-Bus angesprochen werden und ist für einen Temperaturbereich von -40°C bis +120°C geeignet. Die Temperaturmessgenauigkeit liegt bei dem HTS221 bei $\pm 0,5^{\circ}\text{C}$.

Anwendung findet dieser Sensor in Klimaanlage, Heiz- und Belüftungssystemen, Kühlschränken, Luftbefeuchtern oder in der industriellen Automatisierung. [Ard24a]

6.2.5. Digitales Mikrofon (MP34DT05)

Das MP34DT05-A ist ein kompaktes, stromsparendes, omnidirektionales, digitales Mikrofon mit einem kapazitiven Sensorelement und einer IC-Schnittstelle. Das Sensorelement, das in der Lage ist, akustische Wellen zu detektieren, wird mit einem speziellen Silizium-Mikrobearbeitungsverfahren für die Herstellung von Audiosensoren hergestellt. Die IC-Schnittstelle wird in einem speziellen Halbleiterherstellungsverfahren (CMOS-Verfahren[GW22][Ber20]) hergestellt, das die Entwicklung einer speziellen Schaltung ermöglicht, die ein digitales Signal im Pulse Density Modulation (PDM)-Format extern bereitstellen kann. Das Mikrofon hat einen Signal-Rausch-Abstand von 64 dB und eine Empfindlichkeit von -26 dBFS ± 3 dB. Sein maximaler Schalldruckpegel liegt bei 122,5 dB SPL. Der MP34DT05-A ist in einem Surface-Mounted Device (SMD)-kompatiblen, Electromagnetic Interference (EMI)-geschirmten Top-Port-Gehäuse erhältlich und arbeitet zuverlässig über einen erweiterten Temperaturbereich von -40 °C bis +85 °C. Abmessungen des Gehäuses HCLGA-4 LD sind $3 \times 4 \times 1$ (in mm). Anwendung findet das Modul beispielsweise in mobilen Endgeräten, im Bereich der Spracherkennung sowie in Laptops und Notebooks.[STM21][Ard24a]

6.2.6. DC-DC-Wandler (MPM3610)

Das MPM3610-Modul ist ein integriert Gleichspannungs- zu Gleichspannungs-Wandler. Dieser kann Eingangsspannungen von bis zu 21 V verarbeiten mit einem Wirkungsgrad von mindestens 65% bei minimaler Last. Bei einer Eingangsspannung erhöht sich der Wirkungsgrad auf 85%. [Ard24a]

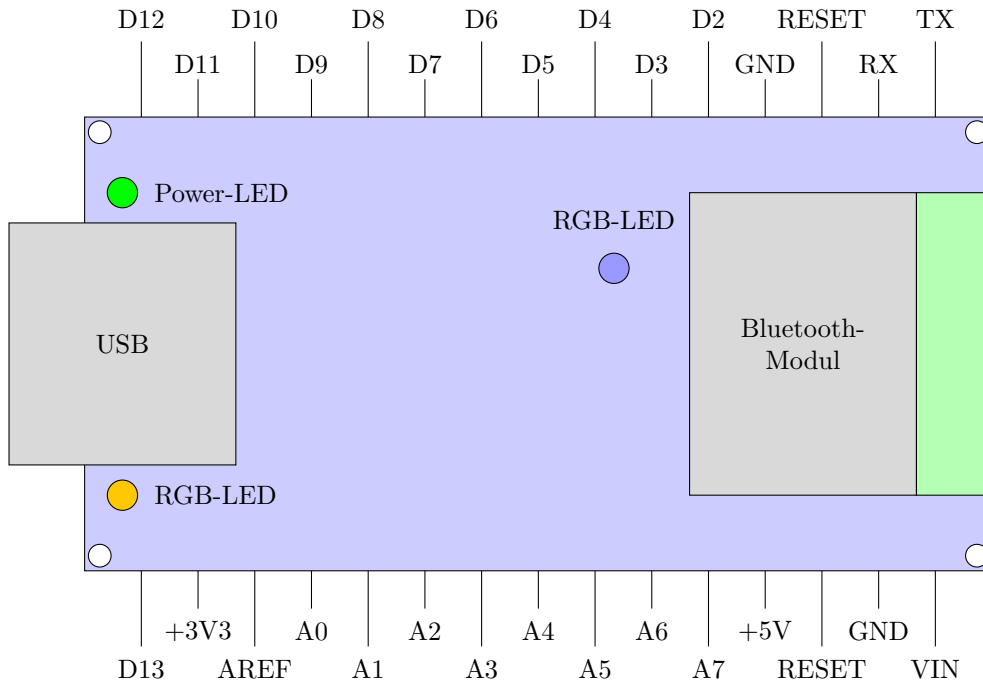


Figure 6.2.: Arduino Nano Pinbelegung [Ard24a]

6.3. Beschreibung der Schnittstellen

Der Arduino Nano 33 BLE-Sense Lite bietet eine Vielzahl von Schnittstellen (siehe Figure 6.2), die das Board mit anderen Komponenten und Geräten einfach verbinden lassen. In diesem Abschnitt sind einige Details zu den wichtigsten Schnittstellen des Boards erläutert.

6.3.1. I²C

Der I²C-Bus ermöglicht die geräteinterne Kommunikation von Mikrocontrollern mit Sensoren, externen Bildschirmen, Echtzeituhren und vielen anderen Bausteinen. Das Kommunikationsprotokoll basiert auf dem Master-Slave-Prinzip. Der Master initiiert und beendet die Kommunikation, stellt den Takt zur Synchronisation bereit und löst Kommunikationsprobleme. Jeder Slave hat eine eindeutige Adresse, mit einer Länge von 7 oder 10 Bit und ermöglicht die Adressierung von bis zu 128 bzw. 1024 unterschiedlichen Slaves in dem selben Bus. Geräte mit unterschiedlichen Adresslängen können im selben Bus koexistieren. Bevor der Master Daten überträgt oder empfängt, muss er einen Slave mit einer vorher vereinbarten Adresse ansprechen.[MS21][STM15][Ber20] Sind ihre Adressen in Übereinstimmung kann im nächsten Schritt ein einzelnes Bit gesendet werden, mit dem dann festgelegt wird, ob der Master Daten an dem Slave übertragen (Binär:1) oder auslesen möchte (Binär:0). Dieser Datenaustausch wird danach bestätigt, sodass weitere Daten ausgetauscht werden können. Zur Beendigung des Datenaustausches wird ein Stop-Signal gesendet.[GW22] In der minimalen Konfiguration werden Master und Slave über die bidirektionalen Busleitungen Serial Data (SDA) und Serial Clock (SCL) verbunden, die über Pull-up-Widerstände an die Versorgungsspannung angeschlossen sind. Weitere Geräte können durch Verbindung ihrer SDA- und SCL-Anschlüsse mit den entsprechenden Busleitungen an den Bus angeschlossen werden.[MS21] In diesem Projekt dient der Arduino als Master und ein zusätzlich angeschlossenes Organic Light Emitting Diode (OLED)-Bildschirm als Slave.

6.3.2. USB

Das Board kann über einen Mini Universal Serial Bus (MUSB) mit einem Computer verbunden werden, um es zu programmieren oder Daten zu übertragen. Die Datenübertragungsrate beträgt dabei 12 Mbit/s. Außerdem kann der Arduino über diesen MUSB-Port Strom beziehen.

6.3.3. Bluetooth®

Die Bluetooth-Verbindung kann als drahtloser Kommunikationsweg eingesetzt werden. Das Bluetooth-Protokoll hat eine Übertragungsrate von 2 Megabit pro Sekunde (Mbps) und eine Sendeleistung von +8 Dezibel Milliwatt (dBm). Die Empfindlichkeit beträgt dabei -95 dBm. Des weiteren verbraucht diese Verbindung im Sendebetrieb 4,8 mA und 4,6 mA im Empfangsbetrieb. Das Bluetooth-Modul ist kompatibel mit mehreren Protokollen, unter anderem mit dem *Thread-Protokoll* und dem *Zigbee-Protokoll*. [Ard24a] [Nor24a]

6.3.4. Weitere Kommunikationsschnittstellen

[Ard24a]

- **NFC-A-Tag:** Near Field Communication (NFC) ist eine zusätzliche Funktion zur drahtlosen Kommunikation über kurze Distanzen. Zudem besitzt der NFC-A-Tag die Funktionen, sich in einen Bereitschaftsmodus versetzen zu lassen, dass durch ein NFC-fähiges Gerät dann initiiert werden kann. Außerdem unterstützt es *touch-to-pair*, diese Funktion ermöglicht eine Kopplung mit anderen NFC-fähigen Geräten durch Berührung.
- **Arm CryptoCell CC310 Security Subsystem:** Für die Durchführung kryptografischer Operationen und Sicherheitsaufgaben. [Nor24c]
- **QSPI/SPI/TWI/I²S/PDM/QDEC:** Verschiedene weitere serielle Kommunikationsschnittstellen, die für den Datenaustausch verwendet werden können.
- **EasyDMA:** Direkt Memory Access (DMA) ist für die Übertragung von Daten zwischen verschiedenen Speicherbereichen, ohne dabei die CPU zu belasten. [GW22]
- **Analog to Digital Converter (ADC):** Wandelt analoge Eingangssignale in digitale Daten um. Der Wandler hat eine Auflösung von 12 Bit und eine maximale Abtastrate von 200 Kilosamples pro Sekunde (ksps).
- **128-Bit-AES/ECB/CCM/AAR-Co-Prozessor:** Co-Prozessor für kryptografische Operationen, der auf dem Advanced Encryption Standard (AES) basiert. Dieser unterstützt verschiedene Betriebsmodi wie Electronic Codebook (ECB), Counter with CBC-MAC (CCM) und Automatic Address Recognition (AAR). [Nor24b]
- **Quad-SPI-Schnittstelle 32 MHz:** SPI-Schnittstelle, die eine maximale Taktrate von 32 MHz unterstützt. Quad-SPI ermöglicht es, Daten schneller als die herkömmliches SPI zu übertragen, indem es vier Datenleitungen verwendet. [Nor23]

6.3.5. Digitale Ein- und Ausgangspins

Das Board verfügt über 14 digitale Ein- und Ausgangspins. Die digitalen Pins können nur zwei Zustände, nach dem Binär-System lesen: wenn ein Spannungssignal vorliegt

und wenn kein Signal vorhanden ist (0 oder 1). Einige der Pins sind zudem zur Pulswweitenmodulation fähig (D3, D5, D6, D9, D10). Außerdem sind die digitalen Pins D11 und D12 als Master-Output-Slave-Input (MOSI) und als Master-Input-Slave-Output (MISO), in einer Serial-Peripheral-Interface (SPI) Kommunikation einsetzbar.[Ard24a]

6.3.6. Analoge Eingangspins

Die Platine hat zusätzlich 8 analoge Eingangspins (A0-A7), die wiederum als Analog-Digital-Wandler (ADC) verwendet werden können. Außerdem sind diese Pins als digitale Ein-/Ausgangspins konfigurierbar. Der Pin A0 kann zudem als Digital-Analog-Wandler (DAC) konfiguriert werden. Die beiden Pins A4 und A5 können außerdem für die I2C-Kommunikation verwendet werden. Dabei fungiert A4 als Datenleitung (SDA), während A5 als Taktleitung (SCL) fungiert.[Ard24a]

6.3.7. Weitere Pins

- **+3,3 V**: Erzeugt interne Stromquelle im Gerät und wird als Referenzspannung verwendet.
- **VIN**: Stromversorgung
- **5V**: Gibt 5V an die externen Komponenten ab.
- **RST-Pin**: Dient zum Zurücksetzen des Arduinos.
- **AREF-Pin**: Liefert die Spannungsreferenz, die der Mikrocontroller zur Zeit verwendet.
[Ard24a]

6.3.8. Reset-Knopf

Der Arduino Nano 33 BLE Sense verfügt über einen Reset-Knopf, welcher das Neustarten der Platine, das Aktivieren des Bootloader-Modus und das Beheben von Problemen (z.B. bei fehlerhaften Programmen) ermöglicht.

Der Reset-Knopf befindet sich auf der Oberseite der Platine, in unmittelbarer Nähe zum USB-Anschluss. Je nach Anzahl und Geschwindigkeit der Tastendrucke führt er unterschiedliche Aktionen aus. Bei einem einzelnen Druck auf die Taste wird ein Systemneustart veranlasst, dabei wird die Stromversorgung kurzzeitig unterbrochen, sodass die Platine und das darauf laufende Programm neu startet. Wenn der Reset-Knopf zweimal schnell hintereinander, innerhalb einer Sekunde gedrückt wird, wird der Bootloader-Modus aktiviert. Dieser ist wichtig, um neue Programme hochzuladen oder das Board wiederherzustellen, falls es durch fehlerhafte Programme blockiert wurde. Die Aktivierung des Bootloader-Modus ist daran zu erkennen, dass die eingebaute LED schnell blinkt.[Ard24a]

6.3.9. LED-Lampen

Im Arduino selbst sind 3 LED's verbaut, die auch alle programmiert werden können. Diese sind vor allem für die Überprüfung der Sensorik oder Softwareprogrammen nützlich, denn sie können je nach Benutzeraktion oder Programmcode blinken, erlöschen oder dauerhaft leuchten. Zu den LED-Lampen gehören:

- Programmierbare Power-LED (grün): Zeigt an, dass das Arduino-Board eingeschaltet ist.
- Programmierbare LED (orange)
- Programmierbare RGB-LED
[Ard24a]

6.4. Hinweis Arduino 33 BLE Sense Lite

Der Arduino Nano 33 BLE Sense Lite ist eine komprimierte Variante vom ursprünglichen Arduino Nano 33 BLE Sense, welcher zusätzlich noch über einen Temperatur- und Feuchtigkeitssensor verfügt. Der Lite hat stattdessen einen Drucksensor integriert, über welchem auch die Temperatur gemessen werden kann, jedoch nicht die Feuchtigkeit.[PA22]

6.5. Bezugsquellen

Als Bezugsquellen dienen vor allem Datenblätter der Hersteller. Die meisten Informationen konnten dem Datenblatt des Arduino entnommen werden, jedoch ist dazu anzumerken, dass sich dieses Datenblatt auf den Arduino 33 BLE Sense bezieht und nicht auf den Arduino 33 BLE Sense *Lite*. Speziell zum *Lite* gibt es jedoch kein Datenblatt, so wurde das Datenblatt vom Arduino 33 BLE Sense herangezogen. Dies stellt sonst kein Problem dar, da sich, wie in 6.4 beschrieben, nur um eine komprimierte Version des Arduino 33 BLE Sense handelt.

Part III.

Arduino Nano 33 BLE Sense - External Sensors and Actors

7. Drehwinkel-Encoder

Der Demonstrator soll mehrere Programme fahren können, deswegen wurde ein Drehwinkel-Encoder der Steuerung hinzugefügt. Dieser wird über drei Pins am Arduino angeschlossen und über zwei weitere Pins wird er mit Spannung versorgt. Durch drehen des Drehschalters werden nacheinander drei Kontakte geschlossen oder geöffnet (ein Kontakt ist immer geschlossen). Dieser dadurch entstehende Signalfluss, bestehend aus zwei um 90 Grad versetzte Sinus bzw. Cosinusschwingungen werden ausgewertet. Daraus wird bestimmt, in welche Richtung (im oder gegen Uhrzeigersinn) und wie weit (inkrementell) gedreht wurde. Mithilfe dieser Logik kann durch ein Menü eine Bewegungsstufe ausgewählt werden, die der Schrittmotor fahren soll.[Bas16] Bei einer Drehung im Uhrzeigersinn wird im Menü eine Bewegungsstufe höher und bei einer Drehung gegen den Uhrzeigersinn eine Bewegungsstufe niedriger ausgewählt. Zusätzlich zum Drehwinkel-Encoder ist auch noch ein Schaltfunktion im Bauteil selbst integriert. Durch eindrücken des Encoders wird ein Taster betätigt, durch den der eingestellte Wert bestätigt und an den Arduino zur weiteren Verarbeitung weitergegeben wird. Zur Besseren Handhabung des Drehwinkel-Encoders wurde noch ein Drehgriff angefertigt und auf dem Drehgeber montiert. Weitere Details:

- **Abmessungen** ($b \times l \times h$): 18 mm $\times$ 31 mm $\times$ 30 mm
- **Betriebsspannung:** 3,3 - 5 V [Sim19]

7.1. Encoder

Die Library Encoder ermöglicht das Zählen von Impulsen aus bestimmten Signalen, die häufig von Drehknöpfen, Motor- oder Wellensensoren sowie anderen Positionsgebern stammen. Diese Bibliothek ist für Arduino und Teensy Boards optimiert und bietet verschiedene Zählmodi, darunter 4X counting für präzise Positionserfassung. Die Verwendung erfolgt durch die Initialisierung eines Encoder-Objekts mit zwei Pins, wobei der erste Pin idealerweise über Interruptfähigkeit verfügt für optimale Leistung. Die Bibliothek unterstützt sowohl I2C- als auch SPI-Schnittstellen und bietet verschiedene Hardware- und Softwareoptionen zur Anpassung an die Anforderungen des Benutzers. Sie wird in diesem Projekt in der Version 1.4.4 verwendet.[Sto24]

7.2. Drehwinkel-Encoder

Um die Funktion und korrekte Ansteuerung des Drehwinkel-Encoders zu testen, wird das Testprogramm 7.1 verwendet. Das Programm wertet die Drehung des Encoders aus und gibt den neuen Zustand im seriellen Monitor der Arduino IDE aus [Sto24].

```
#include <Encoder.h>

const int CLK = 9;
const int DT = 2;
const int SW = 3;
long altePosition = -999;

Encoder meinEncoder(DT, CLK);

void setup()
{
    Serial.begin(9600);
    pinMode(SW, INPUT);
}

void loop()
{
    long neuePosition = meinEncoder.read();

    if (neuePosition != altePosition)
    {
        altePosition = neuePosition;
        Serial.println(neuePosition);
    }

    int StatusTaster = digitalRead(SW);
    if (StatusTaster == 0) {
        Serial.println("Switch_betaetigt");
        Serial.println("Switch_betaetigte");
    }
}
```

Listing 7.1.: Testprogramm für den Encoder

8. Schrittmotor NEMA 17

In diesem Kapitel folgt eine grundsätzliche Beschreibung eines Schrittmotors. Darauf folgt die Erläuterungen zum Aufbau und Funktionsweise eines Schrittmotors sowie die Aufzählung verschiedener Grundbauarten. Nachfolgend werden Ursachen und Lösungen zum Verhindern von Schrittfehlern und der verwendete Schrittmotor genauer erläutert.

8.1. Beschreibung eines Schrittmotors

Ein Schrittmotor ist ein Elektromotor, der sich für präzise Positionierungsaufgaben eignet. Im Gegenteil zu anderen Elektromotoren wird bei einem Schrittmotor keine Positionsmessung oder Positionsregelung benötigt. Andere mechanische Antriebssysteme benötigen einen geschlossenen Regelkreis und mechanische Bremsen um Drehzahl und Position einzuhalten. Der Motor führt durch die Rotation des Rotors diskrete Schritte aus, wobei jeder Steuerimpuls eine Verschiebung um einen konstanten Winkel bewirkt. Mit diesem Motor ist eine erreichbare Positioniergenauigkeit von $0,1^\circ$ möglich. Diese Art von Elektromotor wird beispielsweise in Druckern oder Scanner, aber auch im Kraftfahrzeugbereich verwendet. Im Kraftfahrzeugbereich werden die Schrittmotoren zur Spiegelverstellung sowie der Sitzverstellung verwendet. Das maximal erreichbare Drehmoment eines Schrittmotors liegt bei zwei Newtonmeter und die maximal erreichbare Drehzahl bei ca. 2000 Umdrehung pro Minute. Der Vorteil eines Schrittmotors ist die Wartungsfreiheit, da der Rotor keine Wicklungen hat. [Bab23][Hag21][Ber18][SK21]

8.1.1. Aufbau und Funktionsweise eines Schrittmotors

Der Schrittmotor besteht aus einem Stator, einem Rotor ohne Wicklungen und einer Steuerelektronik. Diese Steuerelektronik setzt sich zusammen aus einer Treiberstufe und der eigentlichen Steuerung. Bei dem Stator handelt es sich um den feststehenden äußeren Teil und bei dem Rotor um den beweglichen inneren Teil, wie in Abbildung 8.1 zu erkennen ist. Im Stator sind Spulen verbaut, die von einem Strom durchflossen werden. Hierdurch entsteht ein magnetisches Feld. Da der Rotor magnetisch ist, folgt er dem Magnetfeld des Stators. Soll eine Bewegung hervorgerufen werden, werden einzelne Wicklungsstränge ein- aus- oder umgeschaltet. Durch diesen Vorgang wird ein rotierendes Magnetfeld erzeugt. Das erzeugte Magnetfeld zieht den Rotor an. Der Rotor wird bei jedem Puls (Takt) um einen Winkelschritt weitergeschaltet. Die Anzahl der Polpaare im Stator geben die Anzahl der Schritte vor. Die Drehzahl und die Drehrichtung hängt von der Reihenfolge und der Häufigkeit der Stromimpulse ab. Es gibt drei verschiedene Betriebsarten, die abhängig von der Genauigkeit und der Drehzahl sind. Im Vollschrittbetrieb werden alle Polpaare bestromt. In dem Halbschrittbetrieb wird die Schrittzahl des Motors verdoppelt, wodurch sich die Positionsauflösung im Gegensatz zum Vollschrittbetrieb verdoppelt. Allerdings wird in diesem Betrieb das Drehmoment reduziert. In dem Mikroschrittbetrieb bewegt sich der Rotor in sehr kleinen Schritten. Hierdurch wird eine hohe Positioniergenauigkeit und ein ruhiger Lauf erreicht, da die Ströme und das Drehmoment in kleineren Schritten verändert werden. Bei einer zu hohen Schrittfrequenz kann ein Schrittverlust entstehen. Bei einem Schrittverlust überspringt der Motor einzelne Winkelschritte und landet in einer vorherigen oder nächsten Position gleicher Phase. Durch die offene Steuerkette werden die Verluste nicht erkannt.[Hag21][Ber18][SK21]

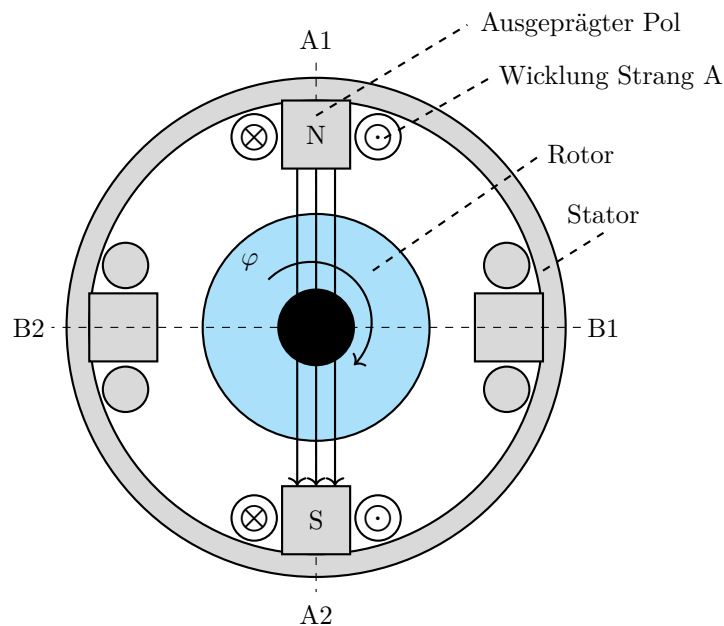


Figure 8.1.: Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]

8.1.2. Schrittmotor Bauformen

Wie bei anderen Elektromotoren gibt es auch bei dem Schrittmotor verschiedene Grundbauarten.

- Permanentmagnetereger-Schrittmotor (PM-Schrittmotor)
- Reluktanzschrittmotor (VR-Schrittmotor)
- Hybridschrittmotor

Der **permanentmagnetische Schrittmotor** hat einen Permanentmagneten in dem Rotor verbaut. Hierbei stellt sich der permanentmagnetische Rotor immer so, dass der Nordpol des Rotors dem Nordpol des Statorfeldes gegenüber liegt und der Südpol des Rotors dem Südpol des Statorfeldes. In dieser Ausrichtung ziehen sich die Pole gegenseitig an. Die Drehrichtung des Rotors hängt von der Fließrichtung des Stromes ab. Der permanentmagnetische Schrittmotor entwickelt im ausgeschalteten Zustand ein Drehmoment zur Selbsthaltung. Dies ist aufgrund des permanentmagnetischen Rotors möglich. Bei diesem Drehmoment handelt es sich um das höchste Drehmoment, das auf die Welle des Motors übertragen werden kann, ohne dass diese sich in eine rotierende Bewegung versetzt. Zu dieser Art von Schrittmotoren gehören beispielsweise der Klauenpol-Schrittmotor und der Scheibenmagnet-Schrittmotor. Bei der zweiten Bauart handelt es sich um den **Reluktanzschrittmotor**. Bei dieser Bauart besteht der Rotor aus einem weichmagnetischen Material und besitzt eine gezahnte Form. So lange der Schrittmotor von keinem Strom durchflossen wird, entsteht kein Magnetfeld. Sobald der Motor in Betrieb genommen wird, entsteht ein magnetischer Fluss innerhalb des Rotors. Wird nun eine Wicklung erregt, wird der nächste Zahn des Rotors angezogen. Dadurch, dass die Rotorzähne ungleich der Polteilung sind, kann das System unendlich lange fortgesetzt werden. Die Anzahl der Schritte und

die Genauigkeit des Reluktanzschrittmotors ist abhängig von der Anzahl der Zähne auf dem Rotor. Aus technischer Sicht sind mit dieser Bauart Schrittwinkel unter 1° möglich. Damit die Drehrichtung verändert werden kann, sind mindestens zwei Strangwicklungen nötig. Bei den **Hybridschrittmotoren**, auch bekannt als Hybridschrittmotoren handelt es sich um eine Kombination aus dem Reluktanzschrittmotor und dem Permanentmagneterreger-Schrittmotor. Durch diese Kombination aus den beiden Schrittmotoren werden die Vorteile aus der kleinen Schrittweite, dem hohen Drehmoment und dem Selbsthaltemoment genutzt. Bei dem Hybridschrittmotor besteht der Rotor aus zwei um eine halbe Zahnteilung versetzten weichmagnetischen Polrädern, die eine zahnförmige Form haben. Bei den beiden Polrädern bildet das eine Polrad den Nordpol und das zweite Polrad den Südpol. Zwischen den beiden Polrädern befindet sich ein Permanentmagnet. Anders als bei anderen Schrittmotoren wird der Rotor bei dieser Bauart axial magnetisiert. Damit ein kleiner Schrittwinkel möglich ist, haben die Statorpole ebenfalls eine zahnförmige Form. Die Ausrichtung des Rotors ist abhängig von der Stromrichtung und wird durch den minimalen Widerstand bestimmt, der sich aus dem Stromfluss durch die einzelnen Stränge ergibt. Wird ein besonders kleiner Schrittwinkel benötigt, kann dies durch Erhöhung der Zähnezahl erreicht werden. [SK21] [Hag21] [Bab23]

8.1.3. Betriebsarten unipolar und bipolar

Neben den oben bereits genannten Betriebsarten, kann außerdem zwischen dem Unipolarbetrieb und dem Bipolarbetrieb unterschieden werden. Der große Unterschied zwischen den beiden Betriebsarten besteht darin, dass in dem Unipolarbetrieb der Strom in eine Richtung fließt. Bei dem Bipolarbetrieb hingegen fließt der Strom in beide Richtungen. Dies ist möglich, da jeder Wicklungsstrang über eine Vollbrücke gespeist wird. Ein weiterer Unterschied besteht in der Schaltung der Zweige, durch die ein Gleichstrom fließt. In dem Unipolarbetrieb werden die beiden Zweige in Reihe geschaltet. Jeder Wicklungsstrang wird mit zwei Drähten parallel gewickelt. Sind die beiden Wicklungsstränge in dem Bipolarbetrieb parallel gewickelt, müssen die Zweige parallelgeschaltet werden. Im Bipolarbetrieb kann ein höherer Wirkungsgrad erzielt werden, wo hingegen der Unipolarbetrieb eine deutlich einfachere Schaltung aufweist. [SK21]

8.1.4. Mikroschrittverfahren

Mit dem Mikroschrittverfahren können Zwischenschritte und Mikroschritte erzeugt werden. Es bietet die Fähigkeit, Geräusche und Vibrationen zu minimieren sowie die Auflösung der Umdrehung zu erhöhen. Um Zwischenschritte zu erreichen, müssen beide Wicklungen eines Schrittmotors gleichzeitig mit Strom versorgt werden. Mit dem Sinus/Cosinus-Mikroschrittverfahren können die Mikroschrittverfahren realisiert werden, indem der Strom in jeder Phase des Motors sinusförmig variiert. Durch die Anwendung sinusförmiger Ströme auf die Motorwicklungen wird eine feinere und gleichmäßigere Bewegung erzielt, was zur einer präziseren Steuerung und reduzierten mechanischen Vibrationen führt. [Mar24]

Ein Mikroschritt-Treiber unterteilt die Motorschrittwinkel in vier oder mehr Teil. Es handelt sich um einen Controller, der die Methode der Stromreglung verwendet, um die Mikroschritte zu erzeugen. Der Controller bietet außerdem den Vorteil einer erhöhten Flexibilität bei der Integration der Strombegrenzung und der Anpassung der Antriebscharakteristik als Reaktion auf Änderungen der Systemdynamik. Die Anzahl der Unterteilungen ist einstellbar, was eine präzise Steuerung und Anpassung an spezifische Anforderungen ermöglicht. [Bab23]

8.2. Schrittverluste verhindern

Um mögliche Schrittverlusten einzugrenzen und oder sie zu verhindern, wird in diesem Kapitel beschrieben, wie die Ursachen für Schrittverluste oder Stillstand methodisch zu ermitteln sind. [Fau20]

8.2.1. Auswahl des Schrittmotors

Zunächst muss ein passender Motor für die Anwendung gewählt werden. Dabei sollten folgende grundlegende Regeln erfüllt sein:

- Motorauswahl durch Höchstwerte für Drehmoment und Drehzahl (Worst-Case-Szenario)
- Verwendung eines Sicherheitsaufschlag von 30 % auf die Drehmoment-Drehzahl-Kennlinie(Kippmoment)
- Sicherstellen, dass externe Ereignisse die Anwendung nicht blockieren können

Sind die geforderten Drehmomente bei den jeweiligen Drehzahlen, den Motorspezifikation entsprechend, dann sind keine Probleme zu erwarten. Ist der Motor zu schwach gewählt und die Anwendung fordert mehr Leistung als der Motor abgeben kann, so bleibt der Motor stehen. Der nächste Schritt ist die Durchführung von Testdurchläufen. Es soll im Betrieb überprüft werden, ob Schrittverluste auftreten. Schrittmotoren verlieren konstruktionsbedingt nicht nur einen einzigen Schritt. Bei geringen Drehzahlen verliert der Motor ein Vielfaches von vier Schritten.[Fau20]

8.2.2. Betriebsart

In diesem Abschnitt werden je nach Betriebsart mögliche Ursachen erläutert, falls der Schrittmotor bei den Tests versagt.

Start-Stopp-Betrieb

Der Motor ist mit der Last fest verbunden und wird mit konstanter Drehzahl betrieben. Innerhalb des ersten Schrittes muss der Motor auf die vorgegebene Frequenz beschleunigen wie in Abbildung 8.2 zu sehen ist.[Fau20]

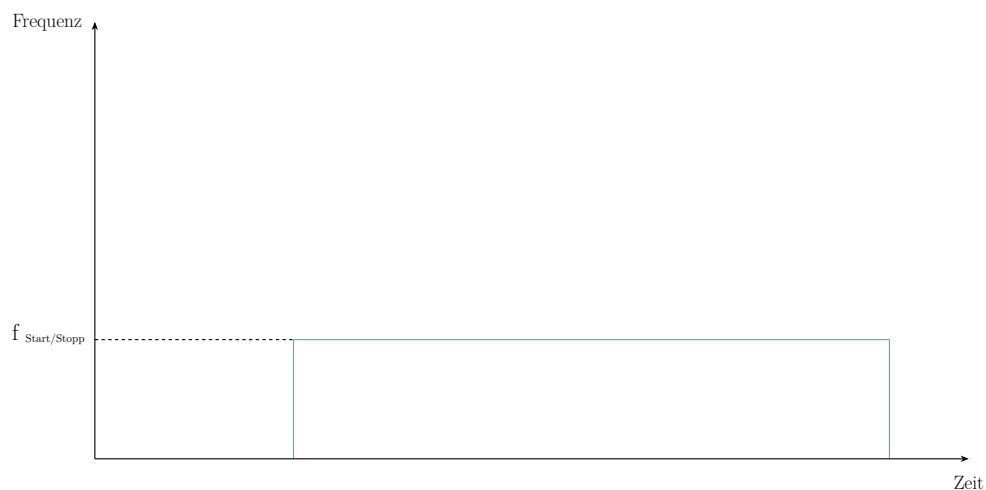


Figure 8.2.: Start-Stopp Frequenz [Fau20]

Fehlerbild: Motor läuft nicht an (siehe Ursachen und Lösungen aus Tabelle 8.1)

Ursachen	Lösungen
Last zu Hoch	Falscher Motor, größeren Motor wählen
Frequenz zu hoch	Frequenz reduzieren
Pendelt der Motor von Links nach Rechts	Es könnte eine Phase unterbrochen oder nicht angeschlossen sein, dies muss repariert werden
Phasenstrom passt nicht	Phasenstrom erhöhen

Table 8.1.: Ursachen und Lösungen: Motor läuft nicht an [Fau20]

Beschleunigung und Rampenprofil (Trapezförmig)

Der Motor kann mit einer im Controller vorgegebenen Beschleunigungsrate bis auf die Maximalfrequenz beschleunigen wie in Abbildung 8.3 zu sehen ist.[Fau20]

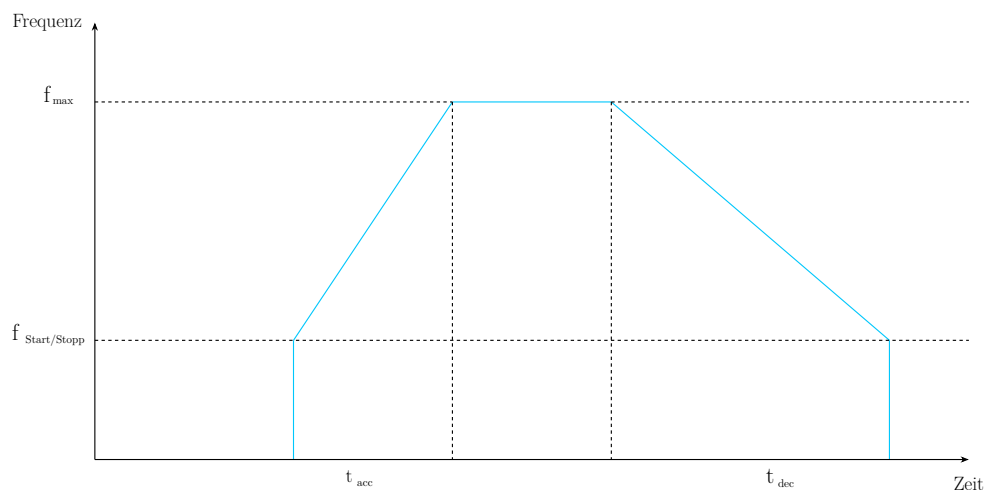


Figure 8.3.: Trapezförmiges Geschwindigkeitsprofil [Fau20]

Fehlerbild: Motor beendet die Beschleunigungsrampe nicht (siehe Ursachen und Lösungen aus Tabelle 8.2)

Ursachen	Lösungen
Motor bleibt bei der Resonanzfrequenz hängen	• Beschleunigung erhöhen, um die Resonanzfrequenz schneller zu durchlaufen • Start-Stopp Frequenz über dem Resonanzpunkt wählen • Halbschritt- oder Mikroschrittbetrieb verwenden • Mechanische Dämpfung vorsehen
Falsche Einstellung von Versorgungsspannung oder Strom zu gering	• Spannung oder Strom erhöhen • Motor mit geringerer Impedanz testen • Stromregelung verwenden
Maximaldrehzahl zu hoch	• Maximaldrehzahl reduzieren • Beschleunigungsrampe abflachen
Schlechte Vorgabe der Beschleunigungsrampe durch Elektronik	• Anderen Controller verwenden

Table 8.2.: Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Fau20]

Fehlerbild: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist (siehe Ursachen und Lösungen aus Tabelle 8.3)

Ursachen	Lösungen
Motor wird an Leistungsgrenze betrieben und bleibt stehen aufgrund zu hoher Beschleunigung	• Ruckeln verringern durch geringere Beschleunigungsrate oder durch unterschiedliche Beschleunigungsrampen, erst steil dann flacher • Drehmoment erhöhen • Motor im Mikroschrittbetrieb betreiben • Mechanische Dämpfung vorsehen

Table 8.3.: Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Fau20]

8.2.3. Externe Ereignisse

Lastrückkopplung

„Manchmal wird der vom Motor angetriebene Mechanismus/Last während der Bewegung „aufgezogen“ und gibt diese Energie wieder an den Motor zurück, wenn die Ströme ausgeschaltet werden. Der Mechanismus könnte z.B. ein Untersetzungsgetriebe sein.“ [Fau20] Wird diese Energie an den Motor zurück geleitet, kann es passieren, dass der Motor sich um einen Winkel verdreht, der mehr als einem Schritt entspricht. Dabei kann es passieren, dass der Motor nicht ausreichend Drehmoment entwickelt und nicht oder erst nach 4 Vollschritten anläuft. [Fau20]

Lösungen:

- Die Kommutierung so programmieren, dass Wert und Polarität vor dem Abschalten gespeichert und beim Wiedereinschalten verwenden werden kann.
- Nicht vollständig den Strom abschalten, sondern bei Motorstillstand einen reduzierten *Stand-By* Strom aufrechterhalten

Erhöhung der Nutzlast mit der Zeit

„Manchmal läuft der Motor für eine lange Zeit störungsfrei und viel später treten die ersten Schrittverluste auf. In diesem Fall ist es sehr wahrscheinlich, dass die Last, die der Motor „sieht“, sich geändert hat. Das kann auf Verschleiß der Motorlager oder ein externes Ereignis zurückzuführen sein.“[Fau20]

Lösungen:

- Prüfen ob ein externes Ereignis durch Veränderung des Mechanismus vorliegt.
- Prüfen ob Lagerverschleiß vorhanden ist. Verwendung von Kugellager erhöhen die Lebensdauer des Motors.
- Verwendung von Schmiermittel um Reibung zu verhindern.

8.3. Beschreibung des verwendeten Schrittmotors

Der hier verwendete NEMA 17 Schrittmotor ist ein bipolarer Schrittmotor. "NEMA" steht hier für die Norm des US-amerikanischen "National Electrical Manufacturers Association". Die 17 in der Modellbezeichnung steht für die Flanschgröße, welche 1,7 x 1,7 Zoll beträgt. Der NEMA 17 besitzt keinen integrierten Treiber, sondern muss über einen externen Treiber wie den DRV8825 angesteuert werden. In Verbindung mit dem Treiber unterstützt der Schrittmotor bis zu 1/32 Mikroschritte.

Der NEMA 17 Schrittmotor findet vor allem in 3D-Druckern, CNC-Fräsen, Kameraschlitten oder Robotikachsen Anwendung.

8.4. Spezifikation

In Tabelle Table 8.4 sind die Spezifikationen des NEMA 17 aufgelistet.

Spezifikation	Wert
Wellenmaße	4,5 x 22 mm (Einzelwelle)
Anschluss	4-Pol Buchse
Schritte pro Umdrehung	200
Abmessungen	42 x 42 x 42 mm
Haltemoment	0,45 Nm
Nennspannung	3,3 V
Nennstrom	1,5 A
Schrittwinkel	1,8°
Anzahl der Phasen	2
Phasenwiderstand	2,2 Ohm
Phaseninduktivität	5 mH
s Isolationswiderstand	500 V DC
Isolationsklasse	B (130°)
Trägheitsmoment	57 g · cm ²
Rastmoment	0,015 Nm
zuläss. Temperaturbereich	-10 °C bis 50 °C

Table 8.4.: Spezifikationen des NEMA 17 Schrittmotors

9. Schrittmotortreiber DRV8825

In diesem Kapitel wird beschrieben, was ein Motortreiber ist, welche Aufgaben er erfüllt und wie man den DRV8825 mit dem Arduino Nano 33 BLE Sense verwendet.

9.1. Allgemeine Beschreibung eines Motortreibers

Ein Motortreiber ist ein Elektronikbaustein, der Mikrocontrollern wie dem Arduino Nano 33 BLE Sense ermöglicht elektrische Motoren anzusteuern. Dabei kann es sich zum Beispiel um Gleichstrommotoren oder Schrittmotoren handeln.

Motortreiber haben mehrere Funktionen. Zum einen verstärken sie die Spannung und den Strom, da Mikrocontroller meist zu wenig Leistung liefern, um den Motor direkt zu betreiben. Zum anderen sorgen Motortreiber dafür, dass man die Drehrichtung und die Drehzahl regeln kann. Außerdem haben viele Motortreiber auch eine Schutzfunktion, die vor Überstrom, Überhitzung oder Kurzschluss schützt.

9.2. Spezifische Beschreibung des Schrittmotortreibers DRV8825

Der DRV8825 ist ein Motortreiber, der vor für bipolare Schrittmotoren wie hier der NEMA 17-04 eingesetzt wird. Er wird in Druckern, CNC-Maschinen oder in der Robotik verwendet.

Der Motor lässt sich über den DRV8825 von Vollschritt bis 1/32-Schritt einstellen. Er verfügt über einen konfigurierbaren Abklingmodus (tritt bei Erreichen des Stromlimits ein), sodass langsames, schnelles oder gemischtes Abklingen verwendet werden kann. Außerdem besitzt er einen stromsparenden Schlafmodus, der die interne Schaltung abschaltet und den Ruhestromverbrauch minimiert.

Der DRV8825 verfügt über Schutzfunktionen für Überstrom, Kurzschluss, Unterspannung und Übertemperatur. Außerdem werden Fehler über den nFAULT-Pin signalisiert.

9.2.1. Anschluss des Treibers mit dem Arduino Nano 33 BLE Sense

Tabelle Table 9.1 kann entnommen werden, wie der DRV8825 an den Arduino Nano 33 BLE Sense angeschlossen wird. Tabelle Table 9.2 zeigt, wie der DRV8825 an den Schrittmotor NEMA 17-04 angeschlossen wird.

DRV8825	Arduino Nano 33 BLE Sense
STEP	z.B. D3
DIR	z.B. D4
EN	GND
nRESET	HIGH (z.B. an 5V)
nSLEEP	HIGH (z.B. an 5V)
GND	GND
VDD	5V

Table 9.1.: Pin-Belegung für den Anschluss des Treibers an den Arduino

DRV8825	Schrittmotor
AOUT1	Spule A, Ende 1
AOUT2	Spule A, Ende 2
BOUT1	Spule B, Ende 1
BOUT2	Spule B, Ende 2

Table 9.2.: Pin-Belegung für den Anschluss des Treibers an den Schrittmotor

9.3. Spezifikationen

Die Spezifikationen sind in Tabelle Table 9.3 und Table 9.4 zu sehen.

Spezifikation	Wert
Versorgungsspannung	-0,3 V bis 47 V
Digitale Eingangsspannung	-0,5 V bis 7 V
Referenzspannung	-0,3 V bis 4 V
Motorstrom	bis zu 2,5 A
Temperaturbereich	-40 °C bis 150 °C

Table 9.3.: Absolute Maximalwerte

Spezifikation	Wert
Versorgungsspannung	8,2 V - 45 V
Referenzspannung	1 V bis 3,5 V
Motorstrom	0 A bis 2,5 A

Table 9.4.: Empfohlene Betriebsbedingungen

9.4. Kalibrierung

Da der Treiber den Strom durch den Schrittmotor regelt ist die Kalibrierung des DRV8825 erforderlich, um zu vermeiden, dass der Schrittmotor überhitzt oder der Treiber überlastet.

Für die Kalibrierung wird ein Multimeter, ein Schraubendreher für das Potentiometer, der GND-Pin des Arduino und der VREF-Messpunkt (Metallplättchen am Potentiometer) benötigt.

Im ersten Schritt ist es wichtig, dass der Schrittmotor nicht angeschlossen ist. Als nächstes wird die Stromversorgung eingeschaltet. Nun wird das Multimeter verwendet, dabei wird der Pluspol am VREF-Messpunkt verbunden und der Minuspol am GND-Anschluss des Arduino. Aus dem abgelesenen Wert kann nun die Zielspannung für VREF berechnet werden und dann am Potentiometer mit dem Schraubendreher eingestellt werden. Dabei ist darauf zu achten das Potentiometer nicht zu überdrehen, da es empfindlich ist.

Nach dem Kalibrieren des Treibers kann der Schrittmotor angeschlossen werden.

9.5. Einfaches Beispiel mit Code

```
// Pin-Zuweisung
const int stepPin = 3;           //STEP
const int dirPin = 4;            //DIR
const int enablePin = 5;         //optional: nENBL (LOW = aktiv)
```

```
void setup() {
  pinMode(stepPin , OUTPUT);
  pinMode(dirPin , OUTPUT);
  pinMode(enablePin , OUTPUT);

  digitalWrite(enablePin , LOW);    //Treiber aktivieren
  digitalWrite(dirPin , HIGH);
  //Drehrichtung setzen (HIGH oder LOW)
}

void loop() {
  // Einen Schritt erzeugen:
  digitalWrite(stepPin , HIGH);
  delayMicroseconds(1000);
  //Schritimpulsdauer (kleiner = schneller)
  digitalWrite(stepPin , LOW);
  delayMicroseconds(1000);
  //Pausenzeit (je nach gewuenschter Drehzahl)
}
```

9.6. Weiterführende Literatur

einfügen

10. Geschwindigkeitssensor LM393

10.1. Allgemeine Beschreibung des Sensors

Der LM393 ist ein einfacher und kostengünstiger Geschwindigkeitssensor, der nach dem Prinzip der optischen Lichtschranke funktioniert. Er besteht aus einem LM393-Komparator, einem Infrarot-Sender-Empfänger-Paar und einem digitalen Ausgang. Der Sensor kann in Kombination mit einer auf einer rotierenden Welle angebrachten Lochscheibe verwendet werden.

10.2. Spezifische Beschreibung des Sensors

Der LM393 Geschwindigkeitssensor besteht aus folgenden Komponenten:

- Infrarot-LED (Sender)
- Fototransistor (Empfänger)
- Lichtschranken-Nut
- LM393 Komparator
- Status-LED
- PCB mit Pins
- Lochscheibe

Wie diese Komponenten die Funktion des Sensors ermöglichen, wird im Folgenden erklärt.

10.2.1. Funktionsweise der Lichtschranke

Die Infrarot-LED und der Fototransistor bilden eine optoelektronische Lichtschranke und sind sich gegenüber auf dem Sensor-PCB angebracht. Die beiden Komponenten sind durch die Lichtschranken-Nut getrennt, welche 5 mm breit ist. In dieser Nut rotiert die Lochscheibe, welche auf der Motorwelle montiert ist und mit 20 Löchern ausgestattet ist. Die Infrarot-LED dient als Sender der Lichtschranke und emittiert kontinuierlich Licht im infraroten Bereich (940 nm) in Richtung des Empfängers. Der Fototransistor ist der Empfänger. Dieser Fototransistor besteht aus einem Halbleitermaterial. Trifft das Infrarotlicht des Senders, durch ein Loch der Lochscheibe auf den Empfänger, so werden dort Elektronen-Loch-Paare erzeugt. Dadurch steigt der Stromfluss durch den Fototransistor. Wird das Infrarotlicht durch die Lochscheibe blockiert sinkt der Strom im Fototransistor.

Die hier verwendete Lochscheibe weist 20 Löcher auf, somit werden pro Umdrehung 20 Signale erzeugt. Da der Sensor aber auf Signaländerungen reagiert werden pro Loch zwei Impulse erzeugt, es werden also 40 Impulse pro Umdrehung gezählt.

10.2.2. Signalverarbeitung mit dem LM393 Komparator

Der LM393 ist ein Komparator, der zwei Analoge Spannungen vergleicht. Eingang A wird mit einer Referenzspannung beschaltet. Eingang B ist mit dem Fototransistor verbunden. Wenn Infrarotlicht auf dem Empfänger auftrifft (durch Loch der Lochscheibe) ändert sich die Spannung am Eingang B. Wenn die Spannung im Fototransistor durch den Lichteinfall höher wird als die Referenzspannung, dann schaltet der Komparator den digitalen Ausgang D0 auf LOW. Wenn der Lichteinfall blockiert ist und somit die Spannung im Fototransistor niedriger ist als die Referenzspannung, dann schaltet der Komparator den digitalen Ausgang D0 auf HIGH. Der LM393 Komparator wird also dazu genutzt kleine Änderungen im Analogsignal zu verstärken und in ein klares digitales Rechtecksignal umzuwandeln.

Am digitalen Ausgang D0 liegt das durch den LM393 erzeugte Signal an. Dieses kann direkt mit einem Mikrocontroller, in diesem Fall einem Arduino Nano 33 BLE Sense verbunden und ausgewertet werden.

10.2.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense

In Tabelle Table 10.1 ist gezeigt wie der Sensor korrekt an den Arduino Nano 33 BLE Sense angeschlossen wird:

Arduino	LM393
5V	VCC
GND	GND
Pin 2	D0

Table 10.1.: Anschluss Speedsensor LM393 an Arduino Nano 33 BLE Sense

10.3. Spezifikationen

Spezifikation	Wert
Modell	Speedsensor LM393 mit Lochscheibe
Versorgungsspannung	3,3 V - 5 V DC
Signalausgabe	Digital
Nutbreite (Lichtschranke)	5 mm
PCB-Abmessungen	32 mm x 14 mm
Lochscheiben-Durchmesser	25,5 mm (20 Löcher)
Besonderheiten	LED-Anzeige, Montageloch

Table 10.2.: Spezifikationen des Speedsensor LM393

10.4. Bibliothek

Für die Verwendung des Sensors mit dem Arduino Nano 33 BLE Sense wird die TimerOne-Bibliothek benötigt. Die TimerOne-Bibliothek wird verwendet, um die Zeitmessung zu ermöglichen, um aus den erzeugten Impulsen des Sensors die Drehzahl zu berechnen.

10.4.1. Installation

Im Folgenden wird erklärt, wie die TimerOne Bibliothek installiert wird.

1. Öffnen der Arduino IDE
2. Klick auf Sketch, Bibliothek einbinden, Bibliotheken verwalten (Abbildung Figure 10.1)
3. In der Suchleiste TimerOne eingeben
4. Klick auf Installieren (Version aus Abbildung Figure 10.2 entnehmen)

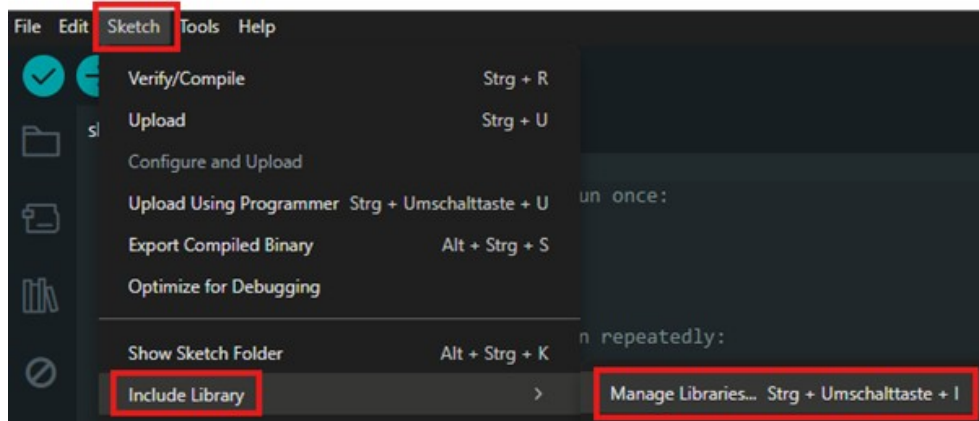


Figure 10.1.: Aufrufen der Bibliotheken in der Arduino IDE



Figure 10.2.: Installation der TimerOne Bibliothek

10.4.2. Code-Beispiel

Im Folgenden ist ein Code-Beispiel gegeben, in dem die verwendeten Funktionen erklärt werden. Für weitere Funktionen der Bibliothek kann man über Datei, Beispiele, TimerOne auf verschiedene Beispiel-Codes zugreifen (Abbildung Figure 10.3).

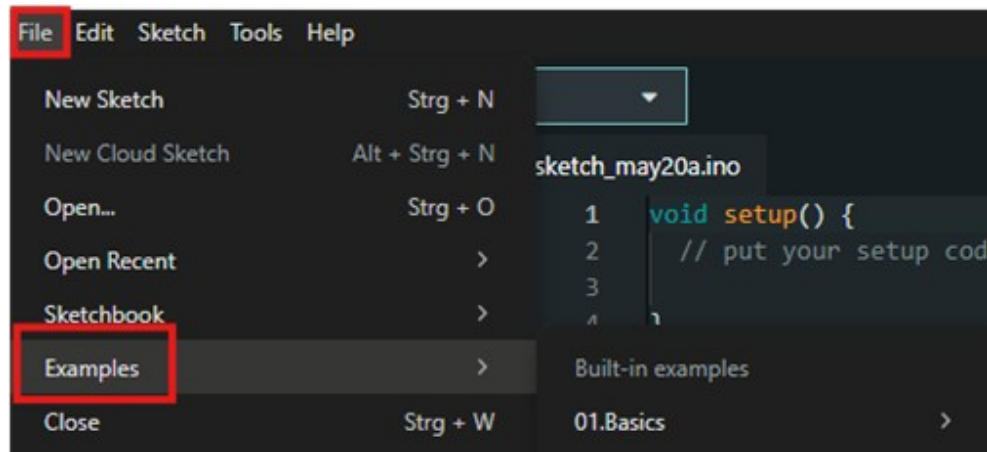


Figure 10.3.: Aufrufen der Beispiele für die TimerOne Bibliothek

10.5. Kalibrierung

Für den LM393 Geschwindigkeitssensor mit Lochscheibe ist keine spezielle Kalibrierung notwendig. Optional kann aber eine empirische Kalibrierung durchgeführt werden, indem man die Umdrehungen mit einem bekannten Drehzahlmesser vergleicht. Bei der Positionierung des Sensors muss darauf geachtet werden, dass die Lichtschranke präzise auf die Mitte der Lochscheibe ausgerichtet ist. Die Lochscheibe muss plan und fest an der Motorachse montiert werden. Außerdem muss die Variable „wheel“ im Code an die Anzahl der Löcher in der Lochscheibe angepasst werden. Da der Sensor auf Flankenwechsel reagiert, muss die Variable auf die doppelte Anzahl der Löcher gesetzt werden. Die Lochscheibe, welche für den Demonstrator verwendet wird, hat 20 Löcher, somit muss „wheel“ auf 40 gesetzt werden, wie in Abbildung xx unter Abschnitt xx zu sehen.

10.6. Einfaches Beispiel mit Code

Das Beispiel geht davon aus, dass die Lochscheibe auf einer sich rotierenden Welle befestigt ist. Mit dem folgenden Code wird die Drehzahl der Welle alle 5 Sekunden im seriellen Monitor ausgegeben.

```
// Import einer Bibliothek
#include "TimerOne.h"
#define pin 2
// benoetigte Variablen
int interval, wheel, counter;
unsigned long previousMicros, usInterval, calc;

void setup() {
  counter = 0; //counter auf 0 setzen
  interval = 5; //5 Sekunden Intervall
  wheel = 20; //Loeher in Encoder-Scheibe

  //Intervall auf eine Minute hoch rechnen
  calc = 60 / interval;
  //Intervallzeit in Mikrosekunden umrechnen
  usInterval = interval * 1000000;
  //Anzahl Loeher Encoderscheibe mal 2 ist Anzahl Signalwechsel
```



```

wheel = wheel * 2;

//Setzen des analogen Pins als Input
pinMode(pin, INPUT);
//Timer auf Intervall initialisieren
Timer1.initialize(usInterval);
attachInterrupt(digitalPinToInterrupt(pin), count, CHANGE);
//fuehrt count aus, wenn sich Signal an Pin 2 aendert

//fuehrt nach Intervall output aus
Timer1.attachInterrupt(output);
Serial.begin(9600);
}

//zaehlen der Encoder-Scheibe
void count() {
  if (micros() - previousMicros >= 700){
    counter++;
    previousMicros = micros();}
}

//Ausgabe im seriellen Monitor

void output() {
  Timer1.detachInterrupt(); //unterbricht Timer
  Serial.print("Drehzahl pro Minute: ");
  int speed = ((counter)*calc) / wheel;
  //Berechnung der Umdrehungen pro Minute

  Serial.println(speed);
  counter = 0;
  //Zuruecksetzen des Zaehlers
  Timer1.attachInterrupt(output);
  //startet Timer erneut
}

void loop() {
  //kein loop notwendig
}

```

10.7. Tests

Um sicherzugehen, dass der Sensor zuverlässig funktioniert, wird empfohlen folgende Tests anhand des einfachen Beispiels mit Code durchzuführen:

- Lichtschrankenprüfung: Drehe die Lochscheibe manuell und beobachte ob die Status-LED bei jedem Loch kurz aufblinkt.
- Signaltest: Starte den Beispielcode und überprüfe ob im eingestellten Zeitintervall Drehzahlen im seriellen Monitor ausgegeben werden.
- Stabilitätstest: Führe längere Messungen durch um sicherzustellen, dass der Sensor stabile Werte liefert.
- Reaktion auf Lichtverhältnisse: Teste den Sensor bei verschiedenen Lichtverhältnissen um externe Störungen der Lichtschranke zu erkennen.

10.8. Weiterführende Literatur

einfügen

11. OLED-Display

In diesem Kapitel wird erklärt wie was ein OLED-Display ist, wie es funktioniert und wie es in diesem Projekt eingebunden wird.

11.1. Allgemeine Beschreibung eines OLED-Displays

Ein OLED-Display ist eine Art von Bildschirm, bei der organische Materialien Licht emittieren, wenn Strom durch sie hindurchfließt. "OLED" steht dabei für "Organic Light Emitting Diode".

Im Gegensatz zu LCD-Displays benötigen OLED-Displays keine Hintergrundbeleuchtung, da jede einzelne Pixelzelle selbst leuchtet. Außerdem sind OLED-Displays oft dünner, leichter, bieten einen besseren Kontrast und einen breiteren Betrachtungswinkel. Durch die schnelle Reaktionszeit von OLED-Displays werden sie vor allem in Smartphones, Fernsehern, Smartwatches oder High-End-Monitoren verbaut.

Nachteile von OLED-Displays liegen darin, dass sie teurer sind als LCD-Displays. Außerdem haben vor allem blaue OLED-Displays oft eine kürzere Lebensdauer und im Allgemeinen besteht die Gefahr des Einbrennens (Burn-in). Das bedeutet, dass es bei statischen Bildern dazu kommen kann, dass Reste des Bildes auf dem Display sichtbar bleiben.

11.2. Spezifische Beschreibung OLED-Displays

Es wird ein 2,42 Zoll OLED-Display von Joy-IT verwendet. Dabei handelt es sich um ein Display mit 128 x 64 Pixeln, welches gelbe Schrift auf schwarzem Hintergrund darstellt. Es besitzt eine SPI- und eine I2C-Schnittstelle, sodass es mit dem verwendeten Arduino Nano 33 BLE Sense kompatibel ist.

11.3. Anschluss des Sensors mit dem Arduino Nano 33 BLE Sense

In Table 11.1 ist gezeigt wie das OLED-Display korrekt an den Arduino Nano 33 BLE Sense angeschlossen wird:

OLED-Display	Arudino Nano 33 BLE Sense
1	GND
2	3V3
4	D9
5	D8
7	A5
8	A4

Table 11.1.: Pinbelegung für I2C-Verbindung zwischen OLED-Display und Arduino Nano 33 BLE Sense

11.4. Spezifikationen

In Table 11.2 sind technische Daten des OLED-Displays aufgelistet.

Spezifikation	Wert
Typ	Negativ OLED
Auflösung	128 x 64 Pixel
Abmessungen	71 x 50 x 7 mm
Versorgungsspannung	3 - 5 V
Versorgungsstrom	90 mA
Logik Level	3 V
High Level Eingang	mind. 2,4 V
Low Level Eingang	max. 0,6 V
Helligkeit	100 - 120 CD/cm ²
Kontrast	> 2000:1
Blickwinkel	> 160°
zuläss. Betriebstemperatur	-40 °C bis 85 °C
Lagerungstemperatur	-45 °C bis 90 °C

Table 11.2.: Spezifikationen des OLED-Displays

11.5. Bibliothek

Für die Verwendung des OLED-Displays mit dem Arduino Nano 33 BLE Sense wird die U8g2 by oliver Bibliothek benötigt.

11.5.1. Installation

Im Folgenden wird erklärt, wie die U8g2 Bibliothek installiert wird.

1. Öffnen der Arduino IDE
2. Klick auf Sketch, Bibliothek einbinden, Bibliotheken verwalten
3. In der Suchleiste U8g2 eingeben
4. Klick auf Installieren (Version aus Figure 11.1 entnehmen)

11.5.2. Code-Beispiel

Auf Code-Beispiele für die U8g2 Bibliothek kann man über Datei, Beispiele, U8g2 zugreifen.

11.6. Einfaches Beispiel mit Code

Mit dem Code-Beispiel kann getestet werden, ob das Display korrekt angeschlossen ist. Wenn die Verbindung korrekt ist, dann erscheint auf dem Display "Hello World".

```
#include <U8g2lib.h>
#include <Wire.h>

U8G2_SSD1309_128X64_NONAME2_1_HW_I2C
u8g2(U8G2_R0; U8X8_PIN_NONE);
```

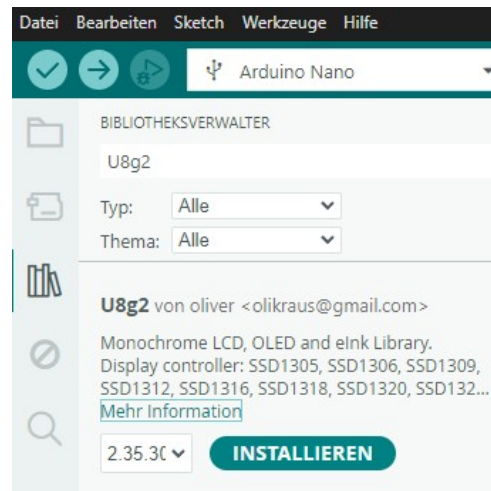


Figure 11.1.: Installation der U8g2 Bibliothek

```
//Initialisieren des Displays
void setup() {
  u8g2.begin();
}

void loop() {
  u8g2.clearBuffer(); //Bildspeicher loeschen
  u8g2.setFont(u8g2_font_ncenB08_tr); //Schriftart waehlen
  u8g2.drawStr(0, 24, "Hello World!"); //Text auf Puffer schreiben
  u8g2.sendBuffer(); //Puffer an Display senden
  delay(1000); //1 Sekunde warten
}
```

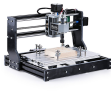
11.7. Weiterführende Literatur

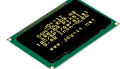
einfügen

Part IV.

Anhang

12. Materialliste

Pos.	Stk.	Bezeichnung	Artikel-Nr.	Preis [€]	Abbildung	Bestelladresse
1	1	Genmitsu 3018 Pro CNC Fräsemaschine	101-60-280PRO	219,00		www.amazon.de
2	1	Arduino Nano 33 BLE Sense Rev.2 nRF52840, mit Header	ARD NANO 33BSH2	47,70		www.reichelt.de
5	1	65 Jumper Wire Kabel im Set	RBS10022	1,55		www.roboter-bausatz.de
4	1	40 Pin Dupont / Jumper Kabel Buchse-Buchse 20 cm	RBS101126	0,98		www.roboter-bausatz.de
5	1	40 Pin Jumper Kabel Buchse-Stecker 20 cm	RBS10028	1,15		www.roboter-bausatz.de
6	1	Arduino - Grove Universal-Kabel, 4-Pin, 20 cm	GRV CA-BLE4PIN20	2,50		www.reichelt.de
7	1	TRU COMPO-NENTS DC-13M Niedervolt-Steckverbinder Stecker, gerade 5.5 mm 2.5 mm 1 St.	1572162 - VQ	3,79		www.conrad.de
8	1	Steckernetzteil, 12 W, 12 V, 1 A	NKSK 200 SW GEW	3,10		www.reichelt.de
9	1	Drucktaster 0,1A-24VAC 1x (Ein) Ø11/9,1mm rt	RAFI 107.301	2,99		www.reichelt.de

10	1	Drucktaster 0,1A-24VAC 1x (Ein) Ø11/9,1mm gn	RAFI 107.507	2,90		www.reichelt.de
11	2	Snap-Action-Mikroschalter, 1x UM, Rollenhebel	AV 32543 AT	1,80		www.reichelt.de
12	1	Entwicklungsboard Schrittmotors- teuerung, A4988	DEBO DRV A4988	5,80		www.reichelt.de
13	1	Entwicklerboards - Display OLED, 2,42", 128x64 Pixel	DEBO OLED 2.42	18,90		www.reichelt.de
14	1	Sunlu PLA Filament 1.75mm 1kg	RBS16624	17,95		www.roboter-bausatz.de
15	1	Experimentier-Slide-Steckboard, 300/100 Kontakte	STECKBOARD S4	4,20		www.reichelt.de
16	1	Kabelbinder-Set, verschiedene Größen, mehrfarbig, 200er-Pack	DELOCK 18628	3,85		www.reichelt.de
17	1	Drehwinkel-Encoder, KY-040	DEBO EN-CODER	2,20		www.reichelt.de
18	1	Wippschalter, 2x Aus, schwarz, I-O	WIPPE 1858.1103	2,60		www.reichelt.de
19	4	Flach-Senkkopfschrauben, Kreuzschlitz, M3, 10 mm	SKS M3X10-50	1,75		www.reichelt.de
20	4	Sechskantmuttern, Edelstahl A2, M3	SK-E M3-100	4,55		www.reichelt.de

21	1	Unterlegscheiben, 10,5 mm	SKU 10,5-100	5,35		www.reichelt. de
22	1	Elektroniklot EL 324 bleifrei 70 g	CFH 52324	10,99		www.reichelt. de

Table 12.1.: Materialliste

Literaturverzeichnis

- [Ard14] Arduino. *Blink: Turn an LED on and off every second*. Ed. by Arduino. 2014. URL: <https://docs.arduino.cc/built-in-examples/basics/Blink/>.
- [Ard24a] Arduino. *ABX00031-Datasheet*. 2024. URL: <https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf>.
- [Ard24b] Arduino. *Downloads: Arduino IDE 2.3.2*. Ed. by Arduino. <https://www.arduino.cc/en/software>, 2024. URL: <https://www.arduino.cc/en/software>.
- [Ard24c] Arduino. *Installing Libraries*. Ed. by Arduino. 2024. URL: <https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-installing-a-library/>.
- [Ard24d] Arduino. *Installing a Board Package in the IDE 2*. Ed. by Arduino. 2024. URL: https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-board-manager/?queryID=145da8e8c0ca68927b79659df14079a5&_gl=1*1c27moy*_ga*MTQ0NDAYMjU0Ni4xNzEyMjI4NjMx*_ga_NEXN8H46L5*MTcxMjkzNTM5NS4xMi4xLjE3MTI5MzYxNTguMCM4wLjE4MTYyNDUyNw.*_fplc*R3haa2kwYmJ5b0owSXBZQmNtNDElMkYySTNJZEZjVDdueVdoViUyQkg1Nm1HTCUyF..
- [Ard24e] Arduino. *loop()*. Ed. by Arduino. 2024. URL: <https://www.arduino.cc/reference/en/language/structure/sketch/loop/>.
- [Ard24f] Arduino. *setup()*. Ed. by Arduino. 2024. URL: <https://www.arduino.cc/reference/en/language/structure/sketch/setup/>.
- [Ard24] Arduino Documentation. *Getting Started with Arduino IDE 2*. 2024. URL: <https://docs.arduino.cc/software/ide-v2/tutorials/getting-started-ide-v2/>.
- [Arm20] Arm. *Arm-Cortex-M4-Processor-Datasheet*. 2020. URL: <https://developer.arm.com/documentation/102832/latest/>.
- [Ava15] Avago Technologies. *Datasheet - APDS-9960 - Digital Proximity, Ambient Light, RGB and Gesture Sensor*. 2015. URL: https://cdn.sparkfun.com/assets/learn_tutorials/3/2/1/Avago-APDS-9960-datasheet.pdf.
- [Bab23] G. Babel. *Elektrische Antriebe in der Fahrzeugtechnik: Lehr- und Arbeitsbuch*. 5. Auflage. Wiesbaden and Heidelberg: Springer Vieweg, 2023. ISBN: 978-3-658-40585-4. DOI: 10.1007/978-3-658-40586-1.
- [Bas16] S. Basler. *Encoder und Motor-Feedback-Systeme*. Wiesbaden: Springer Fachmedien Wiesbaden, 2016. ISBN: 978-3-658-12843-2. DOI: 10.1007/978-3-658-12844-9. URL: <https://link.springer.com/book/10.1007/978-3-658-12844-9>.
- [Ber18] H. Bernstein. *Elektrotechnik/Elektronik für Maschinenbauer: Einfach und praxisgerecht*. 3., überarbeitete Auflage. Lehrbuch. Wiesbaden and Heidelberg: Springer Vieweg, 2018. ISBN: 978-3-658-20837-0. DOI: 10.1007/978-3-658-20838-7.

-
- [Ber20] H. Bernstein. *Mikrocontroller: Grundlagen der Hard- und Software der Mikrocontroller ATtiny2313, ATtiny26 und ATmega32*. 2., aktualisierte und erweiterte Auflage. Lehrbuch. Wiesbaden and Heidelberg: Springer Vieweg, 2020. ISBN: 978-3-658-30067-8.
- [Fau20] Faulhaber Drive Systems. *FAULHABER Tutorial: Schrittverluste verhindern bei Schrittmotoren*. Ed. by DR. FRITZ FAULHABER GMBH & CO. KG. Schönaich · Deutschland, 2020. URL: <https://www.faulhaber.com/de/know-how/tutorials/schrittmotoren-tutorial-schrittverluste-verhindern-bei-schrittmotoren/>.
- [GW22] W. Gehrke and M. Winzker. *Digitaltechnik: Grundlagen, VHDL, FPGAs, Mikrocontroller*. 8. Auflage. Berlin and Heidelberg: Springer Vieweg, 2022. ISBN: 9783662639535. URL: <https://link.springer.com/book/10.1007/978-3-662-63954-2>.
- [Hag21] R. Hagl. *Elektrische Antriebstechnik*. 3., überarbeitete und erweiterte Auflage. München: Hanser, 2021. ISBN: 978-3-446-46572-5. DOI: 10.3139/9783446468214.
- [LD12] J. Lienig and M. Dietrich, eds. *Entwurf integrierter 3D-Systeme der Elektronik*. Berlin and Heidelberg: Springer Vieweg, 2012. ISBN: 978-3-642-30572-6. DOI: 10.1007/978-3-642-30572-6.
- [MS21] A. Meroth and P. Sora. *Sensornetzwerke in Theorie und Praxis*. Wiesbaden: Springer Fachmedien Wiesbaden, 2021. ISBN: 978-3-658-31708-9. DOI: 10.1007/978-3-658-31709-6. URL: <https://link.springer.com/book/10.1007/978-3-658-31709-6>.
- [Mar24] Marc McComb. *Micro-stepping for Stepper Motors*. Ed. by Microchip Technology. 2024.
- [Nor23] Nordic Semiconductor. *nRF5340 Product Specification: QSPI — Quad serial peripheral interface*. 2023. URL: <https://infocenter.nordicsemi.com>.
- [Nor24a] Nordic Semiconductor. *nRF52840 Product Specification 2*. 2024. URL: <https://infocenter.nordicsemi.com>.
- [Nor24b] Nordic Semiconductor. *nRF52840 Product Specification Memory*. 2024. URL: <https://infocenter.nordicsemi.com>.
- [Nor24c] Nordic Semiconductor. *nRF9161 Product Specification: Cryptocell-ARM TrustZone CryptoCell 310*. 2024. URL: <https://infocenter.nordicsemi.com>.
- [PA22] Petr Filipi and Arduino Tech Support Team. *Difference_between_A33BLESense_and_SenseLite: A Difference between A N 33 BLE Sense vs. Sense Lite*. 2022. URL: <https://forum.arduino.cc/t/a-difference-between-a-n-33-ble-sense-vs-sense-lite/1030305>.
- [SK21] D. Schröder and R. Kennel. *Elektrische Antriebe – Grundlagen*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021. ISBN: 978-3-662-63100-3. DOI: 10.1007/978-3-662-63101-0.
- [STM15] STMICROELECTRONICS. *Datasheet - LSM9DS1- iNEMO inertial module: 3D accelerometer, 3D gyroscope, 3D magnetometer*. Ed. by STMICROELECTRONICS. 2015. URL: <https://www.st.com/en/mems-and-sensors/lsm9ds1.html>.
- [STM17] STMICROELECTRONICS. *Datasheet - LPS22HB-MEMS nano pressure sensor: 260-1260 hPa absolute digital output barometer*. Ed. by STMICROELECTRONICS. 2017. URL: <https://www.st.com/en/mems-and-sensors/lps22hb.html>.

- [STM21] STMICROELECTRONICS. *Datasheet - MP34DT05-A - MEMS audio sensor omnidirectional digital microphone*. Ed. by STMICROELECTRONICS. 2021.
- [Sim19] Simac Electronics GmbH. *COM-KY040RE-Datenblatt: Drehencoder mit Tasterfunktion*. [\[pdf\]](#). 2019. URL: www.joy-it.net.
- [Sto24] P. Stoffregen. *Encoder Library*. Ed. by PJRC. [\[pdf\]](#). 2024. URL: https://www.pjrc.com/teensy/td_libs_Encoder.html.

Index

File

- Arduino, 28
- blink.ino, 43, 44
- blnik.ino, 39
- Fade, 44
- File/Examples/01.Basics, 44
- libraries, 30–32
- MySketch, 28
- MySketch.ino, 28
- Nano33BLESenseLE, 30
- Nano33BLESenseLED, 31, 32

Inertial Measurement Unit

- see* IMU, 11, 50

AAR, 11, 53

ADC, 11, 53

Advanced Encryption Standard

- see* AES, 11, 53

AES, 11, 53

Analog to Digital Converter

- see* ADC, 11, 53

Automatic Address Recognition

- see* AAR, 11, 53

BLE, 11, 49, 50, 52

Bluetooth Low Energy

- see* BLE, 11, 49

CCM, 11, 53

Counter with CBC-MAC

- see* CCM, 11, 53

Direkt Memory Access

- see* DMA, 11, 53

DMA, 11, 53

I²C, 11, 50–52

IC, 11, 51

IDE, 11, 44

IMU, 11, 50

Integrated Development Environment

- see* IDE, 11, 44

Inter Circuit

- see* IC, 11, 51

Inter-Integrated Circuit

- see* I²C, 11, 50

LED, 11, 50

Light Emitting Diode

- see* LED, 11, 50

Mini Universal Serial Bus

- see* MSB, 11, 53

MUSB, 11, 53

Near Field Communication

- see* NFC, 11, 53

NFC, 11, 53

OLED, 11, 52

Organic Light Emitting Diode

- see* OLED, 11, 52

PDM, 11, 51

Pulse Density Modulation

- see* PDM, 11, 51

SCL, 11, 52

SDA, 11, 52

Serial Clock

- see* SCL, 11, 52

Serial Data

- see* SDA, 11, 52

Serial Peripheral Interface

- see* SPI, 11, 51

SMD, 11, 51

SoC, 11, 49

SPI, 11, 51, 53

Surface-Mounted Device

- see* SM, 11, 51

System on Chip

- see* SoC, 11, 49

Ultraviolettstrahlung

- see* UV, 11, 50

UV, 11, 50